

# Graph Algorithm Templates

---

`Queue<Integer> queue = new ArrayDeque<>()` throughout.

Six algorithms — each with a canonical template and representative problems.

---

## Complexity Legend

---

|                  |                                          |                                                    |
|------------------|------------------------------------------|----------------------------------------------------|
| BFS / DFS        | $O(V + E)$                               | visit each vertex and edge once                    |
| Topo sort (Kahn) | $O(V + E)$                               | each node enqueued once, each edge relaxed once    |
| Union-Find       | $\sim O(n \cdot \alpha(n)) \approx O(n)$ | $\alpha$ = inverse Ackermann, effectively constant |
| Dijkstra         | $O((V + E) \log V)$                      | non-negative weights ONLY                          |

## Quick Reference Table

| #    | Name                                           | Description                                                                                                                                                                                        | Intuition                                                                                                | Time                                                         | Space          | Key Implementation |
|------|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|----------------|--------------------|
| 994  | Rotting Oranges                                | Grid of <code>0</code> (empty), <code>1</code> (fresh), <code>2</code> (rotten). Each minute rotten oranges infect adjacent fresh ones. Return minutes until no fresh remain, or <code>-1</code> . |                                                                                                          | $O(V+E)$                                                     | $O(V)$         | Standard           |
| 542  | 01 Matrix                                      | For each cell, return the distance to the nearest <code>0</code> .                                                                                                                                 |                                                                                                          | $O(m \cdot n)$                                               | $O(m \cdot n)$ | Standard           |
| 127  | Word Ladder                                    | Minimum number of transformations from <code>beginWord</code> to <code>endWord</code> , changing one letter at a time. Each intermediate word must be in <code>wordList</code> .                   |                                                                                                          | $O(N \cdot L \cdot 26)$ where $N$ = words, $L$ = word length | $O(N \cdot L)$ | Standard           |
| 130  | Surrounded Regions                             | Flip all <code>'0'</code> regions not connected to the border to <code>'X'</code> .                                                                                                                |                                                                                                          | $O(m \cdot n)$                                               | $O(m \cdot n)$ | Standard           |
| 417  | Pacific Atlantic Water Flow                    | Return all cells from which water can flow to both the Pacific (top/left border) and Atlantic (bottom/right border).                                                                               |                                                                                                          | $O(m \cdot n)$                                               | $O(m \cdot n)$ | Standard           |
| 207  | Course Schedule                                | Can you finish all courses given prerequisites? Detect if there is a directed cycle.                                                                                                               |                                                                                                          | $O(V+E)$                                                     | $O(V+E)$       | Standard           |
| 210  | Course Schedule II                             | Return a valid order to take all courses. Return <code>[]</code> if impossible.                                                                                                                    |                                                                                                          | $O(V+E)$                                                     | $O(V+E)$       | Standard           |
| 310  | Minimum Height Trees                           | Find all roots that minimize tree height. At most 2 answers — they are the center node(s) of the longest path.                                                                                     | the best root sits at the center of the longest path; peeling leaves layer by layer converges there.     | $O(V+E)$                                                     | $O(V+E)$       | Standard           |
| 547  | Number of Provinces                            | Count the number of connected components (provinces) in an undirected graph given as an adjacency matrix.                                                                                          | start with <code>n</code> separate sets; every successful merge drops the count by one.                  | $O(n^2 \cdot \alpha(n))$                                     | $O(n)$         | Standard           |
| 684  | Redundant Connection                           | Find the edge that creates a cycle in an undirected graph that would otherwise be a tree.                                                                                                          | if both endpoints are already in the same set, this edge closes a cycle — it's the redundant one.        | $O(n \cdot \alpha(n))$                                       | $O(n)$         | Standard           |
| 1319 | Number of Operations to Make Network Connected | Minimum cable moves to connect all <code>n</code> computers. Return <code>-1</code> if impossible.                                                                                                 | each redundant cable can be moved to bridge one gap, so <code>components - 1</code> moves suffice if you | $O(E \cdot \alpha(n))$                                       | $O(n)$         | Standard           |

| #    | Name                                                  | Description                                                                                                                                            | Intuition                                                                                                                                 | Time                                                 | Space                     | Key Implementation                                                                                                                                                                                                                                                 |
|------|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|      |                                                       |                                                                                                                                                        | have enough spare cables.                                                                                                                 |                                                      |                           |                                                                                                                                                                                                                                                                    |
| 743  | Network Delay Time                                    | Time for a signal from <code>k</code> to reach all <code>n</code> nodes. Return -1 if unreachable.                                                     | every node is reached by its shortest delay; the network is "done" when the slowest of those arrives.                                     | $O((V+E)\log V)$                                     | $O(V+E)$                  | Standard                                                                                                                                                                                                                                                           |
| 787  | Cheapest Flights Within K Stops                       | Cheapest price from <code>src</code> to <code>dst</code> using at most <code>k</code> stops ( <code>k+1</code> edges).                                 | each round lets paths grow by one more edge, so <code>k+1</code> rounds caps the path at <code>k</code> stops.                            | $O(k \cdot E)$                                       | $O(V)$                    | Standard                                                                                                                                                                                                                                                           |
| 1514 | Path with Maximum Probability                         | Find the path from <code>start</code> to <code>end</code> with maximum success probability (product of edge probabilities).                            | probabilities in $[0,1]$ only shrink when multiplied, so the "closest" (highest-probability) node settles first — Dijkstra still applies. | $O((V+E)\log V)$                                     | $O(V+E)$                  | Standard                                                                                                                                                                                                                                                           |
| 785  | Is Graph Bipartite?                                   | Can graph nodes be split into two groups where every edge connects nodes in different groups?                                                          | paint each neighbor the opposite color; a clash means an odd-length cycle, which can't be 2-colored.                                      | $O(V+E)$                                             | $O(V)$                    | Standard                                                                                                                                                                                                                                                           |
| 1584 | Min Cost to Connect All Points                        | Connect all points (on a 2D plane) with minimum total Manhattan distance. Return that total cost.                                                      | grow one tree, always swallowing the cheapest edge that reaches a node not yet in it.                                                     | $O(n^2 \log n)$                                      | $O(n^2)$                  | Standard                                                                                                                                                                                                                                                           |
| 1631 | Path With Minimum Effort                              | Find a path from top-left $[0,0]$ to bottom-right $[m-1,n-1]$ that minimizes the maximum absolute difference between adjacent cells.                   |                                                                                                                                           | $O(m \cdot n \cdot \log(m \cdot n))$                 | $O(m \cdot n)$            | Modified Dijkstra. Instead of summing edge weights, the "cost" of a path is the maximum edge cost. Priority queue ordered by effort; <code>effort[r][c]</code> = minimum over all paths of their max-abs-diff.                                                     |
| 269  | Alien Dictionary                                      | Given a sorted list of alien words, determine the order of characters in the alien alphabet. Return the order as a string, or empty string if invalid. |                                                                                                                                           | $O(C)$ where $C$ = total characters across all words | $O(1)$ (at most 26 nodes) | Build a directed graph of character ordering. Compare each adjacent pair of words to find the first differing character → that gives an edge. Then topological sort. Return empty if there's a cycle or if shorter word is a prefix of longer word in wrong order. |
| 261  | Graph Valid Tree                                      | Given <code>n</code> nodes labeled $0..n-1$ and a list of undirected edges, determine if they form a valid tree (connected, no cycles).                |                                                                                                                                           | $O(n \cdot \alpha(n)) \approx O(n)$                  | $O(n)$                    | A valid tree has exactly <code>n-1</code> edges. Use Union Find to detect cycles. If any edge connects two nodes already in the same component → cycle → not a tree.                                                                                               |
| 323  | Number of Connected Components in an Undirected Graph | Given <code>n</code> nodes and a list of undirected edges, return the number of connected components.                                                  |                                                                                                                                           | $O(n \cdot \alpha(n)) \approx O(n)$                  | $O(n)$                    | decrement on merge                                                                                                                                                                                                                                                 |

| #    | Name                           | Description                                                                                                                                                                       | Intuition                                                                                                                                                                                   | Time                                      | Space          | Key Implementation                                                                                                                                                 |
|------|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 286  | Walls and Gates                | Each cell is a gate (0), wall (-1), or empty (INF = 2147483647). Fill each empty room with distance to its nearest gate.                                                          |                                                                                                                                                                                             | $O(m \cdot n)$                            | $O(m \cdot n)$ | multi-source BFS — seed the queue with every gate at distance 0, then expand outward simultaneously.                                                               |
| 721  | Accounts Merge                 | Each account has a name and a list of emails. Merge accounts that share any email. Return merged accounts with emails sorted.                                                     |                                                                                                                                                                                             | $O(n \cdot a)$ with sorting $O(n \log n)$ | $O(n)$         | Union Find over emails. Assign each unique email an integer id, union all emails within the same account, then group emails by their root.                         |
| 827  | Making A Large Island          | In a binary grid you may change at most one 0 to 1. Return the size of the largest island possible.                                                                               |                                                                                                                                                                                             | $O(m \cdot n)$                            | $O(m \cdot n)$ | two passes. First DFS-color each island with a unique id (starting at 2) and record its size. Then for every 0, sum the sizes of distinct neighboring islands + 1. |
| 909  | Snakes and Ladders             | On an $n \times n$ board numbered in boustrophedon (zigzag) order, find the minimum dice moves (1-6) from square 1 to square $n^2$ . Ladders/snakes teleport you.                 |                                                                                                                                                                                             | $O(n^2)$                                  | $O(n^2)$       | BFS over square numbers. Convert a square number to (row, col) accounting for the zigzag layout.                                                                   |
| 1091 | Shortest Path in Binary Matrix | In an $n \times n$ binary grid, find the length of the shortest clear path (cells of 0) from top-left to bottom-right, moving in 8 directions.                                    |                                                                                                                                                                                             | $O(m \cdot n)$                            | $O(m \cdot n)$ | standard BFS but with 8-directional movement (includes diagonals).                                                                                                 |
| 126  | Word Ladder II                 | Find all shortest transformation sequences from <code>beginWord</code> to <code>endWord</code> , changing one letter at a time, each intermediate word in <code>wordList</code> . | BFS level-by-level builds a parent (predecessor) map of which words lead to which; once <code>endWord</code> is reached, DFS backtracks through parents to reconstruct every shortest path. | $O(N \cdot L \cdot 26)$ BFS + paths       | $O(N \cdot L)$ | Standard                                                                                                                                                           |
| 133  | Clone Graph                    | Deep-copy an undirected graph where each node has a value and a list of neighbors.                                                                                                | Use a map from original node to its clone to avoid re-cloning and to wire neighbors correctly; BFS the graph, creating clones on first sight.                                               | $O(V+E)$                                  | $O(V)$         | Standard                                                                                                                                                           |
| 277  | Find the Celebrity             | Among <code>n</code> people, find the celebrity: known by everyone, knows nobody. <code>knows(a, b)</code> is the API.                                                            | A single linear scan narrows to one candidate (if <code>candidate</code> knows <code>i</code> , the candidate can't be the celebrity, so <code>i</code> becomes the                         | $O(n)$                                    | $O(1)$         | Standard                                                                                                                                                           |

| #   | Name                                 | Description                                                                                                                                 | Intuition                                                                                                                                                                                        | Time                                         | Space          | Key Implementation                                                                                                                                                                     |
|-----|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|     |                                      |                                                                                                                                             | new candidate); a second pass verifies.                                                                                                                                                          |                                              |                |                                                                                                                                                                                        |
| 296 | Best Meeting Point                   | Given a grid where <b>1</b> marks a home, find the minimum total Manhattan distance to a single meeting point.                              | Manhattan distance separates into independent x and y components; the optimal coordinate on each axis is the median of the occupied coordinates.                                                 | $O(m \cdot n)$                               | $O(m \cdot n)$ | Standard                                                                                                                                                                               |
| 305 | Number of Islands II                 | Land cells are added one at a time to a water grid; after each addition return the current number of islands.                               |                                                                                                                                                                                                  | $O(k \cdot \alpha(m \cdot n))$               | $O(m \cdot n)$ | Online Union-Find — when a cell becomes land, union it with any already-land 4-neighbors; track component count.                                                                       |
| 317 | Shortest Distance from All Buildings | Find the empty land cell <b>0</b> minimizing total walking distance to every building <b>1</b> ( <b>2</b> = obstacle).                      | BFS from each building, accumulating distances into every reachable empty cell and counting how many buildings reach it; the answer is the minimum total among cells reachable by all buildings. | $O(B \cdot m \cdot n)$ where $B$ = buildings | $O(m \cdot n)$ | Standard                                                                                                                                                                               |
| 399 | Evaluate Division                    | Given equations $a/b = \text{value}$ , answer queries $c/d$ . Return -1.0 if undeterminable.                                                | Treat variables as nodes and ratios as weighted directed edges; the answer to a query is the product of edge weights along any path from numerator to denominator.                               | $O(Q \cdot (V+E))$                           | $O(V+E)$       | Standard                                                                                                                                                                               |
| 407 | Trapping Rain Water II               | Given a 2D height map, compute how much water it can trap after raining.                                                                    |                                                                                                                                                                                                  | $O(m \cdot n \cdot \log(m \cdot n))$         | $O(m \cdot n)$ | Dijkstra-style BFS from the border inward using a min-heap of boundary heights; water level at a cell is the highest of the minimum boundary it must pass — pop the lowest wall first. |
| 463 | Island Perimeter                     | Given a grid with exactly one island, return its perimeter.                                                                                 | Each land cell contributes 4 sides, minus 2 for every shared edge with an adjacent land cell (counted once per pair).                                                                            | $O(m \cdot n)$                               | $O(1)$         | Standard                                                                                                                                                                               |
| 529 | Minesweeper                          | Reveal a cell on a Minesweeper board. <b>'M'</b> mine, <b>'E'</b> unrevealed empty, <b>'B'</b> revealed blank, digit = adjacent mine count. |                                                                                                                                                                                                  | $O(m \cdot n)$                               | $O(m \cdot n)$ | BFS/DFS flood fill with 8 directions; if the clicked cell is a mine, mark <b>'X'</b> ; otherwise count adjacent mines, and only recurse when count is 0.                               |
| 694 | Number of Distinct Islands           | Count islands that are distinct by shape (translations are the same, rotations are not).                                                    |                                                                                                                                                                                                  | $O(m \cdot n)$                               | $O(m \cdot n)$ | DFS recording the path signature (the direction taken at each step plus a backtrack marker); two islands with                                                                          |

| #   | Name                             | Description                                                                                                                                                                                                                                            | Intuition                                                                                                 | Time                             | Space                            | Key Implementation                                                                                                                                                                         |
|-----|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|----------------------------------|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|     |                                  |                                                                                                                                                                                                                                                        |                                                                                                           |                                  |                                  | identical signatures have the same shape.                                                                                                                                                  |
| 733 | Flood Fill                       | Starting at <code>(sr, sc)</code> , change all connected same-color pixels to <code>color</code> .                                                                                                                                                     |                                                                                                           | $O(m \cdot n)$                   | $O(m \cdot n)$                   | Standard                                                                                                                                                                                   |
| 752 | Open the Lock                    | A 4-wheel lock starts at <code>"0000"</code> ; each move turns one wheel $\pm 1$ . Find minimum moves to reach <code>target</code> , avoiding <code>deadends</code> .                                                                                  |                                                                                                           | $O(10^4 \cdot 8)$                | $O(10^4)$                        | BFS over the 4-digit state space; each state has 8 neighbors (each wheel up or down). Treat deadends as visited.                                                                           |
| 778 | Swim in Rising Water             | At time <code>t</code> you can stand on any cell with elevation $\leq t$ . Find the least time to swim from <code>(0, 0)</code> to <code>(n-1, n-1)</code> .                                                                                           |                                                                                                           | $O(n^2 \cdot \log(n))$           | $O(n^2)$                         | Modified Dijkstra — minimize the maximum elevation along the path (not the sum). Min-heap ordered by current max elevation.                                                                |
| 797 | All Paths From Source to Target  | In a DAG given as adjacency list <code>graph</code> , return all paths from node <code>0</code> to node <code>n-1</code> .                                                                                                                             | Since it is a DAG with no cycles, plain DFS backtracking enumerates every path; no visited set is needed. | $O(2^V \cdot V)$                 | $O(V)$                           | Standard                                                                                                                                                                                   |
| 802 | Find Eventual Safe States        | A node is safe if every path from it eventually reaches a terminal node (no outgoing edges). Return all safe nodes sorted.                                                                                                                             |                                                                                                           | $O(V+E)$                         | $O(V+E)$                         | Topological sort on the reversed graph (Kahn's) — start from terminal nodes (out-degree 0); a node becomes safe once all its outgoing edges lead to safe nodes.                            |
| 815 | Bus Routes                       | <code>routes[i]</code> is the stops served by bus <code>i</code> . Find the minimum number of buses to travel from <code>source</code> to <code>target</code> .                                                                                        |                                                                                                           | $O(\text{sum of route lengths})$ | $O(\text{sum of route lengths})$ | BFS where the "nodes" are buses (routes); build stop→routes index, then BFS route-to-route through shared stops, counting buses taken.                                                     |
| 847 | Shortest Path Visiting All Nodes | Find the length of the shortest path in an undirected graph that visits every node (may revisit nodes/edges).                                                                                                                                          |                                                                                                           | $O(2^n \cdot n^2)$               | $O(2^n \cdot n)$                 | BFS over states <code>(node, visitedMask)</code> where <code>mask</code> is a bitmask of visited nodes; the goal is any state with all bits set. Start BFS from every node simultaneously. |
| 851 | Loud and Rich                    | <code>richer[i] = [a, b]</code> means <code>a</code> is richer than <code>b</code> ; <code>quiet[i]</code> is the quietness of person <code>i</code> . For each person find the quietest person among everyone at least as rich (themselves included). |                                                                                                           | $O(V+E)$                         | $O(V+E)$                         | Topological sort (Kahn's) on the richer→poorer graph; propagate the quietest-known answer from richer people down to poorer ones in topo order.                                            |
| 886 | Possible Bipartition             | Given <code>dislikes</code> pairs, split <code>n</code> people into two groups so disliking people are never in the same group.                                                                                                                        |                                                                                                           | $O(V+E)$                         | $O(V+E)$                         | Standard                                                                                                                                                                                   |

| #    | Name                            | Description                                                                                                                                                              | Intuition | Time                                     | Space            | Key Implementation                                                                                                                                                       |
|------|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|------------------------------------------|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 913  | Cat and Mouse                   | A mouse (start node 1) and cat (start node 2) move on a graph; mouse wins by reaching node 0, cat wins by catching the mouse, else draw. Return 0/1/2 with optimal play. |           | $O(V^3)$                                 | $O(V^2)$         | Game-theory BFS over states (mouse, cat, turn) ; start from known terminal states and propagate results backward (retrograde analysis) by counting undecided moves.      |
| 934  | Shortest Bridge                 | A grid has exactly two islands of 1 s. Find the minimum number of 0 s to flip to connect them.                                                                           |           | $O(n^2)$                                 | $O(n^2)$         | DFS to mark the first island (seeding a BFS queue with its cells), then multi-source BFS expanding outward until reaching the second island; BFS levels = bridge length. |
| 1034 | Coloring A Border               | Color the border cells of the connected component containing (row, col) with color. A border cell touches the grid edge or a cell of a different original color.         |           | $O(m \cdot n)$                           | $O(m \cdot n)$   | BFS over same-color connected cells; mark a cell as a border when it has fewer than 4 same-component neighbors, recolor borders after traversal.                         |
| 1102 | Path With Maximum Minimum Value | Find a path from top-left to bottom-right maximizing the minimum cell value along the path (4-directional).                                                              |           | $O(m \cdot n \cdot \log(m \cdot n))$     | $O(m \cdot n)$   | Modified Dijkstra with a max-heap — the path "score" is the minimum cell value seen; greedily expand the highest-scoring frontier first.                                 |
| 1136 | Parallel Courses                | Courses 1..n with prerequisite relations ; in each semester take all courses whose prerequisites are done. Return minimum semesters, or -1 if impossible.                |           | $O(V+E)$                                 | $O(V+E)$         | Topological sort (Kahn's) processed level-by-level — each BFS level is one semester; if not all courses are processed, a cycle exists.                                   |
| 1162 | As Far from Land as Possible    | In a grid of 0 (water) and 1 (land), find the water cell whose distance to the nearest land is maximized; return that distance, or -1.                                   |           | $O(n^2)$                                 | $O(n^2)$         | Multi-source BFS seeded with all land cells; the last BFS level expanded gives the maximum distance.                                                                     |
| 1197 | Minimum Knight Moves            | On an infinite chessboard, find the minimum knight moves from (0,0) to (x, y) .                                                                                          |           | $O(\max(x,y)^2)$                         | $O(\max(x,y)^2)$ | BFS over knight moves (8 jump offsets); exploit symmetry by working in the first quadrant to bound the visited set.                                                      |
| 1202 | Smallest String With Swaps      | Given index pairs that may be swapped any number of times, return the lexicographically smallest string achievable.                                                      |           | $O(n \cdot \log(n) + E \cdot \alpha(n))$ | $O(n)$           | Union-Find groups swappable indices into connected components; within each component, sort the characters and place them back in ascending index order.                  |
| 1245 | Tree Diameter                   | Given the edges of an undirected tree, return its diameter (the number                                                                                                   |           | $O(V+E)$                                 | $O(V+E)$         | Two BFS passes — BFS from any node finds the farthest node u ; a second BFS from                                                                                         |

| #    | Name                                               | Description                                                                                                 | Intuition | Time          | Space  | Key Implementation                                                                                                                                          |
|------|----------------------------------------------------|-------------------------------------------------------------------------------------------------------------|-----------|---------------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
|      |                                                    | of edges on the longest path between any two nodes).                                                        |           |               |        | u gives the diameter as the maximum distance.                                                                                                               |
| 1293 | Shortest Path in a Grid with Obstacles Elimination | Find the shortest path from (0,0) to (m-1,n-1) in a grid where you may eliminate at most k obstacles (1 s). |           | O(m·n·k)      | O(m·n) | BFS over states (row, col, remainingK); the visited set tracks the best remaining eliminations per cell so a cell can be revisited with more budget.        |
| 1559 | Detect Cycles in 2D Grid                           | Return true if the grid contains a cycle of length ≥ 4 made of the same character.                          |           | O(m·n·α(m·n)) | O(m·n) | Union-Find over grid cells — for each cell union it with its up and left same-character neighbors; if a union finds them already connected, a cycle exists. |
| 1743 | Restore the Array From Adjacent Pairs              | Given all adjacent pairs of an array (in any order), reconstruct the original array.                        |           | O(n)          | O(n)   | Build an adjacency graph; the two endpoints have a single neighbor. Start from an endpoint and walk the path, avoiding the previous node.                   |

## Graph Representation

```
// Adjacency list (most common)
List<List<Integer>> graph = new ArrayList<>();
for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
for (int[] edge : edges) {
    graph.get(edge[0]).add(edge[1]);
    graph.get(edge[1]).add(edge[0]);    // omit for directed
}

// Weighted adjacency list: int[] = {neighbor, weight}
List<List<int[]>> graph = new ArrayList<>();
for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
for (int[] edge : edges)
    graph.get(edge[0]).add(new int[]{edge[1], edge[2]});

// Grid 4-directional neighbors
int[] dr = {1, -1, 0, 0};
int[] dc = {0, 0, 1, -1};
```



## All Six Templates

```
// 1. BFS – shortest path / level order
// MENTAL MODEL: explore in rings of equal distance, so the first time you reach a node is the shortest
// WHEN: "fewest steps", "shortest path" on an unweighted graph
Queue<Integer> queue = new ArrayDeque<>();
boolean[] visited = new boolean[n];
queue.offer(start);
visited[start] = true;
while (!queue.isEmpty()) {
    int node = queue.poll();
    for (int neighbor : graph.get(node)) {
        if (!visited[neighbor]) {
            visited[neighbor] = true;
            queue.offer(neighbor);
        }
    }
}

// 2. TOPOLOGICAL SORT – Kahn's BFS
// MENTAL MODEL: peel off nodes with no remaining prerequisites; if any get stuck, a cycle exists.
// WHEN: "valid order", "dependencies/prerequisites", "detect cycle" on a directed graph
int[] inDegree = new int[n];
for (int[] edge : edges) inDegree[edge[1]]++;
Queue<Integer> queue = new ArrayDeque<>();
for (int i = 0; i < n; i++) if (inDegree[i] == 0) queue.offer(i);
List<Integer> order = new ArrayList<>();
while (!queue.isEmpty()) {
    int node = queue.poll();
    order.add(node);
    for (int neighbor : graph.get(node))
        if (--inDegree[neighbor] == 0) queue.offer(neighbor);
}
// order.size() == n → no cycle

// 3. UNION FIND
// MENTAL MODEL: each set points to one representative; merge sets by linking roots, query by finding r
// WHEN: "connected components", "is it already connected", "merge groups dynamically"
int[] parent, rank;
void init(int n) {
    parent = new int[n]; rank = new int[n];
    for (int i = 0; i < n; i++) parent[i] = i;
}
int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]); // path compression
    return parent[x];
}
boolean union(int x, int y) {
    int px = find(x), py = find(y);
    if (px == py) return false;
    if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
    parent[py] = px;
    if (rank[px] == rank[py]) rank[px]++;
    return true;
}

// 4. DIJKSTRA – shortest path (non-negative weights)
// MENTAL MODEL: greedily settle the closest unfinished node; non-negative weights guarantee it's final
// WHEN: "shortest/cheapest path" with non-negative weights
int[] dist = new int[n];
Arrays.fill(dist, Integer.MAX_VALUE);
dist[src] = 0;
```

```

PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]); // min-heap on cost
pq.offer(new int[]{src, 0});
while (!pq.isEmpty()) {
    int[] current = pq.poll();
    int node = current[0], d = current[1];
    if (d > dist[node]) continue; // stale entry
    for (int[] nb : graph.get(node)) {
        int next = nb[0], w = nb[1];
        if (dist[node] + w < dist[next]) {
            dist[next] = dist[node] + w;
            pq.offer(new int[]{next, dist[next]});
        }
    }
}

// 5. BIPARTITE CHECK – BFS 2-coloring
// MENTAL MODEL: paint neighbors the opposite color; a neighbor that's already your color means an odd
// WHEN: "split into two groups", "2-colorable", "no edge within a group"
int[] color = new int[n];
Arrays.fill(color, -1);
Queue<Integer> queue = new ArrayDeque<>();
for (int start = 0; start < n; start++) {
    if (color[start] != -1) continue;
    color[start] = 0;
    queue.offer(start);
    while (!queue.isEmpty()) {
        int node = queue.poll();
        for (int neighbor : graph.get(node)) {
            if (color[neighbor] == -1) { color[neighbor] = 1 - color[node]; queue.offer(neighbor); }
            else if (color[neighbor] == color[node]) return false; // conflict
        }
    }
}
return true;

// 6. PRIM'S MST – greedy, always add cheapest edge to growing tree
// MENTAL MODEL: grow one tree outward, each step absorbing the cheapest edge that reaches a new node.
// WHEN: "connect all nodes at minimum total cost"
boolean[] inMST = new boolean[n];
int[] minEdge = new int[n];
Arrays.fill(minEdge, Integer.MAX_VALUE);
minEdge[0] = 0;
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
pq.offer(new int[]{0, 0});
int total = 0;
while (!pq.isEmpty()) {
    int[] current = pq.poll();
    int node = current[0], cost = current[1];
    if (inMST[node]) continue;
    inMST[node] = true;
    total += cost;
    for (int[] nb : graph.get(node))
        if (!inMST[nb[0]] && nb[1] < minEdge[nb[0]]) {
            minEdge[nb[0]] = nb[1];
            pq.offer(new int[]{nb[0], nb[1]});
        }
}

```

# Part 1 — BFS Shortest Path

## Single-Source vs Multi-Source

Single-source: one starting node, find distances to all others

Multi-source: seed queue with ALL starting nodes at once → BFS fans out from all simultaneously  
→ avoids nested loops, handles "nearest X" questions in  $O(n)$

### #994 Rotting Oranges

**Description:** Grid of 0 (empty), 1 (fresh), 2 (rotten). Each minute rotten oranges infect adjacent fresh ones. Return minutes until no fresh remain, or -1.

**Algorithm:** Multi-source BFS — seed queue with all rotten cells simultaneously.

```
class Solution {
    public int orangesRotting(int[][] grid) {
        int rows = grid.length, cols = grid[0].length;
        Queue<int[]> queue = new ArrayDeque<>();
        int fresh = 0;
        for (int r = 0; r < rows; r++)
            for (int c = 0; c < cols; c++) {
                if (grid[r][c] == 2) queue.offer(new int[]{r, c});
                if (grid[r][c] == 1) fresh++;
            }
        int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};
        int minutes = 0;
        while (!queue.isEmpty() && fresh > 0) {
            minutes++;
            int size = queue.size();
            for (int i = 0; i < size; i++) {
                int[] current = queue.poll();
                for (int d = 0; d < 4; d++) {
                    int nr = current[0]+dr[d], nc = current[1]+dc[d];
                    if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && grid[nr][nc] == 1) {
                        grid[nr][nc] = 2;
                        queue.offer(new int[]{nr, nc});
                        fresh--;
                    }
                }
            }
        }
        return fresh == 0 ? minutes : -1;
    }
}
```

**Time**  $O(V+E)$  | **Space**  $O(V)$

### #542 01 Matrix

**Description:** For each cell, return the distance to the nearest 0.

**Algorithm:** Multi-source BFS from all 0 cells simultaneously. Initialize `dist[r][c] = MAX` for 1 cells.

```

class Solution {
    public int[][] updateMatrix(int[][] mat) {
        int rows = mat.length, cols = mat[0].length;
        int[][] dist = new int[rows][cols];
        Queue<int[]> queue = new ArrayDeque<>();
        for (int r = 0; r < rows; r++)
            for (int c = 0; c < cols; c++) {
                if (mat[r][c] == 0) {
                    queue.offer(new int[]{r, c});
                } else {
                    dist[r][c] = Integer.MAX_VALUE;
                }
            }
        int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};
        while (!queue.isEmpty()) {
            int[] current = queue.poll();
            for (int d = 0; d < 4; d++) {
                int nr = current[0]+dr[d], nc = current[1]+dc[d];
                if (nr >= 0 && nr < rows && nc >= 0 && nc < cols
                    && dist[current[0]][current[1]] + 1 < dist[nr][nc]) {
                    dist[nr][nc] = dist[current[0]][current[1]] + 1;
                    queue.offer(new int[]{nr, nc});
                }
            }
        }
        return dist;
    }
}

```

**Time**  $O(m \cdot n)$  | **Space**  $O(m \cdot n)$

## #127 Word Ladder

**Description:** Minimum number of transformations from `beginWord` to `endWord` , changing one letter at a time. Each intermediate word must be in `wordList` .

**Algorithm:** BFS on word states. Remove words from set when visited to prevent re-visiting.

```

class Solution {
    public int ladderLength(String beginWord, String endWord, List<String> wordList) {
        Set<String> wordSet = new HashSet<>(wordList);
        if (!wordSet.contains(endWord)) return 0;
        Queue<String> queue = new ArrayDeque<>();
        queue.offer(beginWord);
        wordSet.remove(beginWord);
        int steps = 1;
        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int i = 0; i < size; i++) {
                char[] word = queue.poll().toCharArray();
                for (int j = 0; j < word.length; j++) {
                    char original = word[j];
                    for (char c = 'a'; c <= 'z'; c++) {
                        word[j] = c;
                        String next = new String(word);
                        if (next.equals(endWord)) return steps + 1;
                        if (wordSet.contains(next)) {
                            wordSet.remove(next);
                            queue.offer(next);
                        }
                    }
                    word[j] = original;
                }
            }
            steps++;
        }
        return 0;
    }
}

```

**Time**  $O(N \cdot L \cdot 26)$  where  $N$  = words,  $L$  = word length | **Space**  $O(N \cdot L)$

## #130 Surrounded Regions

**Description:** Flip all '0' regions not connected to the border to 'X'.

**Algorithm:** Multi-source BFS from all border '0' cells. Mark reachable as safe ('S'), then flip all remaining '0' to 'X' and restore 'S' to '0'.

```

class Solution {
    public void solve(char[][] board) {
        int rows = board.length, cols = board[0].length;
        Queue<int[]> queue = new ArrayDeque<>();
        for (int r = 0; r < rows; r++)
            for (int c = 0; c < cols; c++)
                if ((r == 0 || r == rows-1 || c == 0 || c == cols-1) && board[r][c] == '0') {
                    board[r][c] = 'S';
                    queue.offer(new int[]{r, c});
                }
        int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};
        while (!queue.isEmpty()) {
            int[] current = queue.poll();
            for (int d = 0; d < 4; d++) {
                int nr = current[0]+dr[d], nc = current[1]+dc[d];
                if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && board[nr][nc] == '0') {
                    board[nr][nc] = 'S';
                    queue.offer(new int[]{nr, nc});
                }
            }
        }
        for (int r = 0; r < rows; r++)
            for (int c = 0; c < cols; c++)
                board[r][c] = board[r][c] == 'S' ? '0' : 'X';
    }
}

```

**Time**  $O(m \cdot n)$  | **Space**  $O(m \cdot n)$

## #417 Pacific Atlantic Water Flow

**Description:** Return all cells from which water can flow to both the Pacific (top/left border) and Atlantic (bottom/right border).

**Algorithm:** Two separate multi-source BFS — one from each ocean's border cells, moving **uphill** (water flows down, so reachable cells going up = cells that drain down to ocean). Answer = intersection of both reachable sets.

```

class Solution {
    int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};

    public List<List<Integer>> pacificAtlantic(int[][] h) {
        int rows = h.length, cols = h[0].length;
        boolean[][] pac = new boolean[rows][cols];
        boolean[][] atl = new boolean[rows][cols];
        Queue<int[]> pacQ = new ArrayDeque<>(), atlQ = new ArrayDeque<>();
        for (int r = 0; r < rows; r++) {
            pac[r][0] = atl[r][cols-1] = true;
            pacQ.offer(new int[]{r, 0});
            atlQ.offer(new int[]{r, cols-1});
        }
        for (int c = 0; c < cols; c++) {
            pac[0][c] = atl[rows-1][c] = true;
            pacQ.offer(new int[]{0, c});
            atlQ.offer(new int[]{rows-1, c});
        }
        bfs(h, pacQ, pac);
        bfs(h, atlQ, atl);
        List<List<Integer>> result = new ArrayList<>();
        for (int r = 0; r < rows; r++)
            for (int c = 0; c < cols; c++)
                if (pac[r][c] && atl[r][c]) result.add(Arrays.asList(r, c));
        return result;
    }

    private void bfs(int[][] h, Queue<int[]> queue, boolean[][] visited) {
        while (!queue.isEmpty()) {
            int[] current = queue.poll();
            for (int d = 0; d < 4; d++) {
                int nr = current[0]+dr[d], nc = current[1]+dc[d];
                if (nr >= 0 && nr < h.length && nc >= 0 && nc < h[0].length
                    && !visited[nr][nc] && h[nr][nc] >= h[current[0]][current[1]]) {
                    visited[nr][nc] = true;
                    queue.offer(new int[]{nr, nc});
                }
            }
        }
    }
}

```

**Time**  $O(m \cdot n)$  | **Space**  $O(m \cdot n)$

## Part 2 — Topological Sort (Kahn's BFS)

**Core idea:** repeatedly remove nodes with `inDegree == 0` (no prerequisites). If all nodes removed  $\rightarrow$  no cycle. Post-order in DFS = reverse of `order` here.

### #207 Course Schedule

**Description:** Can you finish all courses given prerequisites? Detect if there is a directed cycle.

```

class Solution {
    public boolean canFinish(int numCourses, int[][] prerequisites) {
        List<List<Integer>> graph = new ArrayList<>();
        int[] inDegree = new int[numCourses];
        for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());
        for (int[] p : prerequisites) {
            graph.get(p[1]).add(p[0]);
            inDegree[p[0]]++;
        }
        Queue<Integer> queue = new ArrayDeque<>();
        for (int i = 0; i < numCourses; i++) if (inDegree[i] == 0) queue.offer(i);
        int processed = 0;
        while (!queue.isEmpty()) {
            int node = queue.poll();
            processed++;
            for (int neighbor : graph.get(node))
                if (--inDegree[neighbor] == 0) queue.offer(neighbor);
        }
        return processed == numCourses;    // all processed → no cycle
    }
}

```

**Time**  $O(V+E)$  | **Space**  $O(V+E)$

## #210 Course Schedule II

**Description:** Return a valid order to take all courses. Return `[]` if impossible.

```

class Solution {
    public int[] findOrder(int numCourses, int[][] prerequisites) {
        List<List<Integer>> graph = new ArrayList<>();
        int[] inDegree = new int[numCourses];
        for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());
        for (int[] p : prerequisites) {
            graph.get(p[1]).add(p[0]);
            inDegree[p[0]]++;
        }
        Queue<Integer> queue = new ArrayDeque<>();
        for (int i = 0; i < numCourses; i++) if (inDegree[i] == 0) queue.offer(i);
        int[] order = new int[numCourses];
        int idx = 0;
        while (!queue.isEmpty()) {
            int node = queue.poll();
            order[idx++] = node;
            for (int neighbor : graph.get(node))
                if (--inDegree[neighbor] == 0) queue.offer(neighbor);
        }
        return idx == numCourses ? order : new int[]{};
    }
}

```

**Time**  $O(V+E)$  | **Space**  $O(V+E)$

## #310 Minimum Height Trees

**Description:** Find all roots that minimize tree height. At most 2 answers — they are the center node(s) of the longest path.



**Algorithm:** Iteratively trim leaves (degree-1 nodes) until  $\leq 2$  nodes remain — same as topo sort but on undirected tree.

**Intuition:** the best root sits at the center of the longest path; peeling leaves layer by layer converges there.

```
class Solution {
    public List<Integer> findMinHeightTrees(int n, int[][] edges) {
        if (n == 1) return Collections.singletonList(0);
        List<Set<Integer>> graph = new ArrayList<>();
        for (int i = 0; i < n; i++) graph.add(new HashSet<>());
        for (int[] e : edges) { graph.get(e[0]).add(e[1]); graph.get(e[1]).add(e[0]); }
        Queue<Integer> queue = new ArrayDeque<>();
        for (int i = 0; i < n; i++) if (graph.get(i).size() == 1) queue.offer(i);
        int remaining = n;
        while (remaining > 2) {
            int size = queue.size();
            remaining -= size;
            for (int i = 0; i < size; i++) {
                int leaf = queue.poll();
                int neighbor = graph.get(leaf).iterator().next();
                graph.get(neighbor).remove(leaf);
                if (graph.get(neighbor).size() == 1) queue.offer(neighbor);
            }
        }
        return new ArrayList<>(queue);
    }
}
```

**Time**  $O(V+E)$  | **Space**  $O(V+E)$

---

## 207 vs 210 — Side by Side

|             | #207 (boolean) | #210 (order)                  |
|-------------|----------------|-------------------------------|
| Track:      | int processed  | int[] order, int idx          |
| Return:     | processed == n | idx == n ? order : []         |
| Difference: | just count     | also record node into order[] |

---

## Part 3 — Union Find

**Template (always the same three methods):**

```

int[] parent, rank;

void init(int n) {
    parent = new int[n];
    rank = new int[n];
    for (int i = 0; i < n; i++) parent[i] = i;
}

int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]); // path compression
    return parent[x];
}

boolean union(int x, int y) { // returns false if already connected
    int px = find(x), py = find(y);
    if (px == py) return false;
    if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
    parent[py] = px;
    if (rank[px] == rank[py]) rank[px]++;
    return true;
}

```

## #547 Number of Provinces

**Description:** Count the number of connected components (provinces) in an undirected graph given as an adjacency matrix.

**Intuition:** start with  $n$  separate sets; every successful merge drops the count by one.

```

class Solution {
    int[] parent, rank;
    public int findCircleNum(int[][] isConnected) {
        int n = isConnected.length;
        parent = new int[n]; rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
        int components = n;
        for (int i = 0; i < n; i++)
            for (int j = i+1; j < n; j++)
                if (isConnected[i][j] == 1 && union(i, j)) components--;
        return components;
    }
    int find(int x) { return parent[x] == x ? x : (parent[x] = find(parent[x])); }
    boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;
        if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
        parent[py] = px;
        if (rank[px] == rank[py]) rank[px]++;
        return true;
    }
}

```

**Time**  $O(n^2 \cdot \alpha(n))$  | **Space**  $O(n)$

## #684 Redundant Connection

**Description:** Find the edge that creates a cycle in an undirected graph that would otherwise be a tree.

**Key:** try to union each edge in order. The first edge where both nodes already share a root is the redundant one.

**Intuition:** if both endpoints are already in the same set, this edge closes a cycle — it's the redundant one.

```

class Solution {
    int[] parent, rank;
    public int[] findRedundantConnection(int[][] edges) {
        int n = edges.length;
        parent = new int[n+1]; rank = new int[n+1];
        for (int i = 0; i <= n; i++) parent[i] = i;
        for (int[] edge : edges)
            if (!union(edge[0], edge[1])) return edge; // already connected → cycle
        return new int[]{};
    }
    int find(int x) { return parent[x] == x ? x : (parent[x] = find(parent[x])); }
    boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;
        if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
        parent[py] = px;
        if (rank[px] == rank[py]) rank[px]++;
        return true;
    }
}

```

**Time**  $O(n \cdot \alpha(n))$  | **Space**  $O(n)$

## #1319 Number of Operations to Make Network Connected

**Description:** Minimum cable moves to connect all  $n$  computers. Return -1 if impossible.

**Key:** need at least  $n-1$  edges for  $n$  nodes. Count extra edges (cycles); count components. Answer = components - 1 if enough extras exist.

**Intuition:** each redundant cable can be moved to bridge one gap, so components - 1 moves suffice if you have enough spare cables.

```

class Solution {
    int[] parent, rank;
    public int makeConnected(int n, int[][] connections) {
        if (connections.length < n - 1) return -1; // not enough cables
        parent = new int[n]; rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
        int components = n;
        for (int[] c : connections) if (union(c[0], c[1])) components--;
        return components - 1;
    }
    int find(int x) { return parent[x] == x ? x : (parent[x] = find(parent[x])); }
    boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;
        if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
        parent[py] = px;
        if (rank[px] == rank[py]) rank[px]++;
        return true;
    }
}

```

**Time**  $O(E \cdot \alpha(n))$  | **Space**  $O(n)$

## Part 4 — Dijkstra / Bellman-Ford

### #743 Network Delay Time

**Description:** Time for a signal from `k` to reach all `n` nodes. Return -1 if unreachable.

**Algorithm:** Dijkstra from source `k`. Answer = max of all shortest distances.

**Intuition:** every node is reached by its shortest delay; the network is "done" when the slowest of those arrives.

```
class Solution {
    public int networkDelayTime(int[][] times, int n, int k) {
        List<List<int[]>> graph = new ArrayList<>();
        for (int i = 0; i <= n; i++) graph.add(new ArrayList<>());
        for (int[] t : times) graph.get(t[0]).add(new int[]{t[1], t[2]});
        int[] dist = new int[n+1];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[k] = 0;
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
        pq.offer(new int[]{k, 0});
        while (!pq.isEmpty()) {
            int[] current = pq.poll();
            int node = current[0], d = current[1];
            if (d > dist[node]) continue;
            for (int[] nb : graph.get(node)) {
                int next = nb[0], w = nb[1];
                if (dist[node] + w < dist[next]) {
                    dist[next] = dist[node] + w;
                    pq.offer(new int[]{next, dist[next]});
                }
            }
        }
        int maxDist = 0;
        for (int i = 1; i <= n; i++) {
            if (dist[i] == Integer.MAX_VALUE) return -1;
            maxDist = Math.max(maxDist, dist[i]);
        }
        return maxDist;
    }
}
```

**Time**  $O((V+E)\log V)$  | **Space**  $O(V+E)$

### #787 Cheapest Flights Within K Stops

**Description:** Cheapest price from `src` to `dst` using at most `k` stops (`k+1` edges).

**Algorithm:** Bellman-Ford with exactly `k+1` relaxation rounds. Use a copy `temp[]` each round to prevent using the same edge twice in one round.

**Intuition:** each round lets paths grow by one more edge, so `k+1` rounds caps the path at `k` stops.

```

class Solution {
    public int findCheapestPrice(int n, int[][] flights, int src, int dst, int k) {
        int[] dist = new int[n];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[src] = 0;
        for (int round = 0; round <= k; round++) {           // k stops = k+1 edges
            int[] temp = Arrays.copyOf(dist, n);
            for (int[] f : flights) {
                int u = f[0], v = f[1], w = f[2];
                if (dist[u] != Integer.MAX_VALUE && dist[u] + w < temp[v])
                    temp[v] = dist[u] + w;
            }
            dist = temp;
        }
        return dist[dst] == Integer.MAX_VALUE ? -1 : dist[dst];
    }
}

```

**Time**  $O(k \cdot E)$  | **Space**  $O(V)$

## #1514 Path with Maximum Probability

**Description:** Find the path from `start` to `end` with maximum success probability (product of edge probabilities).

**Algorithm:** Dijkstra with **max-heap** (maximize probability instead of minimize distance). Multiply probabilities instead of adding weights.

**Intuition:** probabilities in  $[0,1]$  only shrink when multiplied, so the "closest" (highest-probability) node settles first — Dijkstra still applies.

```

class Solution {
    public double maxProbability(int n, int[][] edges, double[] succProb, int start, int end) {
        List<List<double[]>> graph = new ArrayList<>();
        for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
        for (int i = 0; i < edges.length; i++) {
            int u = edges[i][0], v = edges[i][1];
            graph.get(u).add(new double[]{v, succProb[i]});
            graph.get(v).add(new double[]{u, succProb[i]});
        }
        double[] prob = new double[n];
        prob[start] = 1.0;
        PriorityQueue<double[]> pq = new PriorityQueue<>((a, b) -> Double.compare(b[1], a[1])); // max-
        pq.offer(new double[]{start, 1.0});
        while (!pq.isEmpty()) {
            double[] current = pq.poll();
            int node = (int) current[0];
            double p = current[1];
            if (node == end) return p;
            if (p < prob[node]) continue;
            for (double[] nb : graph.get(node)) {
                int next = (int) nb[0];
                double newProb = p * nb[1];
                if (newProb > prob[next]) {
                    prob[next] = newProb;
                    pq.offer(new double[]{next, newProb});
                }
            }
        }
        return 0.0;
    }
}

```

**Time**  $O((V+E)\log V)$  | **Space**  $O(V+E)$

## Dijkstra vs Bellman-Ford

|              | Dijkstra                                    | Bellman-Ford                     |
|--------------|---------------------------------------------|----------------------------------|
| Weights:     | Non-negative only                           | Any (handles negative)           |
| Constraint:  | —                                           | k-hop limit (run k+1 rounds)     |
| Time:        | $O((V+E) \log V)$                           | $O(k \cdot E)$                   |
| When to use: | Standard shortest path                      | k-stop limit or negative weights |
| Key trick:   | stale check: $d > \text{dist}[\text{node}]$ | copy array each round (temp[])   |

## Part 5 — Bipartite Check

### #785 Is Graph Bipartite?

**Description:** Can graph nodes be split into two groups where every edge connects nodes in different groups?

**Algorithm:** BFS 2-coloring. If any neighbor has the same color as the current node → not bipartite.

**Key:** run BFS from every unvisited node (graph may be disconnected).

**Intuition:** paint each neighbor the opposite color; a clash means an odd-length cycle, which can't be 2-colored.

```
class Solution {
    public boolean isBipartite(int[][] graph) {
        int n = graph.length;
        int[] color = new int[n];
        Arrays.fill(color, -1);
        Queue<Integer> queue = new ArrayDeque<>();
        for (int start = 0; start < n; start++) {
            if (color[start] != -1) continue;
            color[start] = 0;
            queue.offer(start);
            while (!queue.isEmpty()) {
                int node = queue.poll();
                for (int neighbor : graph[node]) {
                    if (color[neighbor] == -1) {
                        color[neighbor] = 1 - color[node];
                        queue.offer(neighbor);
                    } else if (color[neighbor] == color[node]) {
                        return false;
                    }
                }
            }
        }
        return true;
    }
}
```

**Time**  $O(V+E)$  | **Space**  $O(V)$

# Part 6 — Minimum Spanning Tree (Prim's)

## #1584 Min Cost to Connect All Points

**Description:** Connect all points (on a 2D plane) with minimum total Manhattan distance. Return that total cost.

**Algorithm:** Prim's MST — always add the cheapest edge from the growing tree to a new node.

**Key:** `minEdge[i]` tracks the cheapest known edge from any tree node to node `i`. Update lazily via priority queue; skip nodes already in MST.

**Intuition:** grow one tree, always swallowing the cheapest edge that reaches a node not yet in it.

```
class Solution {
    public int minCostConnectPoints(int[][] points) {
        int n = points.length;
        boolean[] inMST = new boolean[n];
        int[] minEdge = new int[n];
        Arrays.fill(minEdge, Integer.MAX_VALUE);
        minEdge[0] = 0;
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
        pq.offer(new int[]{0, 0});
        int total = 0;
        while (!pq.isEmpty()) {
            int[] current = pq.poll();
            int node = current[0], cost = current[1];
            if (inMST[node]) continue;
            inMST[node] = true;
            total += cost;
            for (int next = 0; next < n; next++) {
                if (!inMST[next]) {
                    int dist = Math.abs(points[node][0] - points[next][0])
                        + Math.abs(points[node][1] - points[next][1]);
                    if (dist < minEdge[next]) {
                        minEdge[next] = dist;
                        pq.offer(new int[]{next, dist});
                    }
                }
            }
        }
        return total;
    }
}
```

**Time**  $O(n^2 \log n)$  | **Space**  $O(n^2)$

# Algorithm Chooser

| Signal in problem                       | Algorithm                 |
|-----------------------------------------|---------------------------|
| Shortest path, unweighted / unit weight | BFS                       |
| Shortest path, non-negative weights     | Dijkstra                  |
| Shortest path, k-hop limit              | Bellman-Ford (k+1 rounds) |
| Shortest path, negative weights         | Bellman-Ford (n-1 rounds) |
| Connected components, cycle detection   | Union Find                |
| Directed cycle / valid ordering         | Topological Sort (Kahn's) |
| 2-colorable graph                       | Bipartite BFS coloring    |
| Connect all nodes minimum cost          | Prim's / Kruskal's MST    |
| "nearest X for all cells"               | Multi-source BFS          |

## Stale Entry Pattern (Dijkstra)

```
int[] current = pq.poll();
int node = current[0], d = current[1];
if (d > dist[node]) continue; // ← this node was already relaxed to a better distance
```

Without this check, stale entries in the heap cause re-processing and wrong results.

It replaces a `visited[]` array — simpler and correct because a better distance always wins.

## Part 7 — Modified Dijkstra / Topological Sort on Chars / Union Find Variants

### #1631 Path With Minimum Effort

**Description:** Find a path from top-left `[0,0]` to bottom-right `[m-1,n-1]` that minimizes the maximum absolute difference between adjacent cells.

**Variation:** Modified Dijkstra. Instead of summing edge weights, the "cost" of a path is the maximum edge cost. Priority queue ordered by effort; `effort[r][c]` = minimum over all paths of their max-abs-diff.



```

class Solution {
    public int minimumEffortPath(int[][] heights) {
        int m = heights.length, n = heights[0].length;
        int[][] effort = new int[m][n];
        for (int[] row : effort) Arrays.fill(row, Integer.MAX_VALUE);
        effort[0][0] = 0;
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
        pq.offer(new int[]{0, 0, 0}); // [effort, row, col]
        int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};
        while (!pq.isEmpty()) {
            int[] current = pq.poll();
            int e = current[0], r = current[1], c = current[2];
            if (r == m-1 && c == n-1) return e;
            if (e > effort[r][c]) continue; // stale entry
            for (int dir = 0; dir < 4; dir++) {
                int nr = r + dr[dir], nc = c + dc[dir];
                if (nr >= 0 && nr < m && nc >= 0 && nc < n) {
                    int newEffort = Math.max(e, Math.abs(heights[nr][nc] - heights[r][c])); // ← VARIA
                    if (newEffort < effort[nr][nc]) {
                        effort[nr][nc] = newEffort;
                        pq.offer(new int[]{newEffort, nr, nc});
                    }
                }
            }
        }
        return 0;
    }
}

```

**Time**  $O(m \cdot n \cdot \log(m \cdot n))$  | **Space**  $O(m \cdot n)$

## #269 Alien Dictionary

**Description:** Given a sorted list of alien words, determine the order of characters in the alien alphabet. Return the order as a string, or empty string if invalid.

**Variation:** Build a directed graph of character ordering. Compare each adjacent pair of words to find the first differing character  $\rightarrow$  that gives an edge. Then topological sort. Return empty if there's a cycle or if shorter word is a prefix of longer word in wrong order.

```

class Solution {
    public String alienOrder(String[] words) {
        Map<Character, Set<Character>> graph = new HashMap<>();
        Map<Character, Integer> inDegree = new HashMap<>();
        for (String word : words)
            for (char c : word.toCharArray()) {
                graph.putIfAbsent(c, new HashSet<>());
                inDegree.putIfAbsent(c, 0);
            }
        for (int i = 0; i < words.length - 1; i++) {
            String w1 = words[i], w2 = words[i+1];
            if (w1.length() > w2.length() && w1.startsWith(w2)) return ""; // ← VARIATION: invalid pre
            for (int j = 0; j < Math.min(w1.length(), w2.length()); j++) {
                if (w1.charAt(j) != w2.charAt(j)) {
                    if (!graph.get(w1.charAt(j)).contains(w2.charAt(j))) {
                        graph.get(w1.charAt(j)).add(w2.charAt(j));
                        inDegree.merge(w2.charAt(j), 1, Integer::sum);
                    }
                    break;
                }
            }
        }
        Queue<Character> queue = new ArrayDeque<>();
        for (char c : inDegree.keySet()) if (inDegree.get(c) == 0) queue.offer(c);
        StringBuilder result = new StringBuilder();
        while (!queue.isEmpty()) {
            char c = queue.poll();
            result.append(c);
            for (char next : graph.get(c)) {
                inDegree.merge(next, -1, Integer::sum);
                if (inDegree.get(next) == 0) queue.offer(next);
            }
        }
        return result.length() == inDegree.size() ? result.toString() : ""; // ← cycle check
    }
}

```

**Time**  $O(C)$  where  $C$  = total characters across all words | **Space**  $O(1)$  (at most 26 nodes)

## #261 Graph Valid Tree

**Description:** Given  $n$  nodes labeled  $0..n-1$  and a list of undirected edges, determine if they form a valid tree (connected, no cycles).

**Variation:** A valid tree has exactly  $n-1$  edges. Use Union Find to detect cycles. If any edge connects two nodes already in the same component  $\rightarrow$  cycle  $\rightarrow$  not a tree.

```

class Solution {
    int[] parent, rank;

    public boolean validTree(int n, int[][] edges) {
        if (edges.length != n - 1) return false; // ← VARIATION: tree must have exactly n-1 edges
        parent = new int[n]; rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
        for (int[] edge : edges)
            if (!union(edge[0], edge[1])) return false; // ← cycle detected → not a tree
        return true;
    }

    int find(int x) { return parent[x] == x ? x : (parent[x] = find(parent[x])); }

    boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;
        if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
        parent[py] = px;
        if (rank[px] == rank[py]) rank[px]++;
        return true;
    }
}

```

**Time**  $O(n \cdot \alpha(n)) \approx O(n)$  | **Space**  $O(n)$

## #323 Number of Connected Components in an Undirected Graph

**Description:** Given  $n$  nodes and a list of undirected edges, return the number of connected components.

**Algorithm:** Union Find. Start with  $n$  components. Each successful union (merging two previously separate components) decrements the count by 1.

```

class Solution {
    int[] parent, rank;

    public int countComponents(int n, int[][] edges) {
        parent = new int[n]; rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
        int components = n; // ← start with n isolated components
        for (int[] edge : edges)
            if (union(edge[0], edge[1])) components--; // ← VARIATION: decrement on merge
        return components;
    }

    int find(int x) { return parent[x] == x ? x : (parent[x] = find(parent[x])); }

    boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;
        if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
        parent[py] = px;
        if (rank[px] == rank[py]) rank[px]++;
        return true;
    }
}

```

**Time**  $O(n \cdot \alpha(n)) \approx O(n)$  | **Space**  $O(n)$

## #286 Walls and Gates

**Description:** Each cell is a gate (0), wall (-1), or empty (INF = 2147483647). Fill each empty room with distance to its nearest gate. **Variation:** multi-source BFS — seed the queue with every gate at distance 0, then expand outward simultaneously.

```
class Solution {
    public void wallsAndGates(int[][] rooms) {
        if (rooms.length == 0) return;
        int m = rooms.length, n = rooms[0].length;
        Queue<int[]> queue = new ArrayDeque<>();
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                if (rooms[i][j] == 0) queue.offer(new int[]{i, j}); // ← VARIATION: seed ALL gates
        int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};
        while (!queue.isEmpty()) {
            int[] cell = queue.poll();
            for (int dir = 0; dir < 4; dir++) {
                int ni = cell[0] + dr[dir], nj = cell[1] + dc[dir];
                if (ni < 0 || ni >= m || nj < 0 || nj >= n || rooms[ni][nj] != Integer.MAX_VALUE) continue;
                rooms[ni][nj] = rooms[cell[0]][cell[1]] + 1;
                queue.offer(new int[]{ni, nj});
            }
        }
    }
}
```

**Time**  $O(m \cdot n)$  | **Space**  $O(m \cdot n)$

## #721 Accounts Merge

**Description:** Each account has a name and a list of emails. Merge accounts that share any email. Return merged accounts with emails sorted. **Variation:** Union Find over emails. Assign each unique email an integer id, union all emails within the same account, then group emails by their root.

```

class Solution {
    private int[] parent;
    public List<List<String>> accountsMerge(List<List<String>> accounts) {
        Map<String, Integer> emailToId = new HashMap<>();
        Map<String, String> emailToName = new HashMap<>();
        int id = 0;
        for (List<String> account : accounts)
            for (int i = 1; i < account.size(); i++) {
                String email = account.get(i);
                if (!emailToId.containsKey(email)) emailToId.put(email, id++);
                emailToName.put(email, account.get(0));
            }
        parent = new int[id];
        for (int i = 0; i < id; i++) parent[i] = i;
        for (List<String> account : accounts)
            for (int i = 2; i < account.size(); i++)
                union(emailToId.get(account.get(1)), emailToId.get(account.get(i))); // ← VARIATION: u
        Map<Integer, List<String>> groups = new HashMap<>();
        for (String email : emailToId.keySet())
            groups.computeIfAbsent(find(emailToId.get(email)), x -> new ArrayList<>()).add(email);
        List<List<String>> result = new ArrayList<>();
        for (List<String> emails : groups.values()) {
            Collections.sort(emails);
            List<String> merged = new ArrayList<>();
            merged.add(emailToName.get(emails.get(0)));
            merged.addAll(emails);
            result.add(merged);
        }
        return result;
    }
    private int find(int x) { return parent[x] == x ? x : (parent[x] = find(parent[x])); }
    private void union(int x, int y) { parent[find(x)] = find(y); }
}

```

**Time**  $O(n \cdot \alpha)$  with sorting  $O(n \log n)$  | **Space**  $O(n)$

## #827 Making A Large Island

**Description:** In a binary grid you may change at most one 0 to 1. Return the size of the largest island possible. **Variation:** two passes. First DFS-color each island with a unique id (starting at 2) and record its size. Then for every 0, sum the sizes of distinct neighboring islands + 1.

```

class Solution {
    private int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};
    public int largestIsland(int[][] grid) {
        int n = grid.length, id = 2;
        Map<Integer, Integer> size = new HashMap<>();
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (grid[i][j] == 1) size.put(id, dfs(grid, i, j, id++)); // ← VARIATION: color + reco
        int max = size.values().stream().mapToInt(x -> x).max().orElse(0);
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (grid[i][j] == 0) {
                    Set<Integer> seen = new HashSet<>();
                    int total = 1;
                    for (int dir = 0; dir < 4; dir++) {
                        int ni = i + dr[dir], nj = j + dc[dir];
                        if (ni < 0 || ni >= n || nj < 0 || nj >= n || grid[ni][nj] <= 1) continue;
                        if (seen.add(grid[ni][nj])) total += size.get(grid[ni][nj]); // ← VARIATION: s
                    }
                    max = Math.max(max, total);
                }
        return max;
    }
    private int dfs(int[][] grid, int i, int j, int id) {
        if (i < 0 || i >= grid.length || j < 0 || j >= grid.length || grid[i][j] != 1) return 0;
        grid[i][j] = id;
        int count = 1;
        for (int dir = 0; dir < 4; dir++) count += dfs(grid, i + dr[dir], j + dc[dir], id);
        return count;
    }
}

```

**Time**  $O(m \cdot n)$  | **Space**  $O(m \cdot n)$

## #909 Snakes and Ladders

**Description:** On an  $n \times n$  board numbered in boustrophedon (zigzag) order, find the minimum dice moves (1-6) from square 1 to square  $n^2$ . Ladders/snakes teleport you. **Variation:** BFS over square numbers. Convert a square number to (row, col) accounting for the zigzag layout.

```

class Solution {
    public int snakesAndLadders(int[][] board) {
        int n = board.length;
        boolean[] visited = new boolean[n * n + 1];
        Queue<Integer> queue = new ArrayDeque<>();
        queue.offer(1);
        visited[1] = true;
        int moves = 0;
        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int s = 0; s < size; s++) {
                int current = queue.poll();
                if (current == n * n) return moves;
                for (int k = 1; k <= 6 && current + k <= n * n; k++) {
                    int next = current + k;
                    int[] rc = toCoord(next, n); // ← VARIATION: number → zigzag coordinate
                    if (board[rc[0]][rc[1]] != -1) next = board[rc[0]][rc[1]]; // snake/ladder
                    if (!visited[next]) { visited[next] = true; queue.offer(next); }
                }
            }
            moves++;
        }
        return -1;
    }
    private int[] toCoord(int num, int n) {
        int row = (num - 1) / n, col = (num - 1) % n;
        if (row % 2 == 1) col = n - 1 - col; // ← VARIATION: reverse on odd rows
        return new int[]{n - 1 - row, col};
    }
}

```

**Time**  $O(n^2)$  | **Space**  $O(n^2)$

## #1091 Shortest Path in Binary Matrix

**Description:** In an  $n \times n$  binary grid, find the length of the shortest clear path (cells of 0) from top-left to bottom-right, moving in 8 directions. **Variation:** standard BFS but with 8-directional movement (includes diagonals).

```

class Solution {
    public int shortestPathBinaryMatrix(int[][] grid) {
        int n = grid.length;
        if (grid[0][0] == 1 || grid[n-1][n-1] == 1) return -1;
        int[] dr = {1,-1,0,0,1,1,-1,-1}, dc = {0,0,1,-1,1,-1,1,-1}; // ← VARIATION: 8 directions
        Queue<int[]> queue = new ArrayDeque<>();
        queue.offer(new int[]{0, 0});
        grid[0][0] = 1;
        int path = 1;
        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int s = 0; s < size; s++) {
                int[] cell = queue.poll();
                if (cell[0] == n-1 && cell[1] == n-1) return path;
                for (int dir = 0; dir < 8; dir++) {
                    int ni = cell[0] + dr[dir], nj = cell[1] + dc[dir];
                    if (ni < 0 || ni >= n || nj < 0 || nj >= n || grid[ni][nj] != 0) continue;
                    grid[ni][nj] = 1;
                    queue.offer(new int[]{ni, nj});
                }
            }
            path++;
        }
        return -1;
    }
}

```

**Time**  $O(m \cdot n)$  | **Space**  $O(m \cdot n)$

## Additional Reference Problems

### #126 Word Ladder II

**Description:** Find all shortest transformation sequences from `beginWord` to `endWord`, changing one letter at a time, each intermediate word in `wordList`.

**Intuition:** BFS level-by-level builds a parent (predecessor) map of which words lead to which; once `endWord` is reached, DFS backtracks through parents to reconstruct every shortest path.

**Algorithm:** BFS recording all predecessors per word; stop expanding once `endWord` found at a level; then DFS from `endWord` back to `beginWord`.



```

class Solution {
    public List<List<String>> findLadders(String beginWord, String endWord, List<String> wordList) {
        List<List<String>> result = new ArrayList<>();
        Set<String> wordSet = new HashSet<>(wordList);
        if (!wordSet.contains(endWord)) return result;
        Map<String, List<String>> parents = new HashMap<>();
        Set<String> level = new HashSet<>();
        level.add(beginWord);
        boolean found = false;
        while (!level.isEmpty() && !found) {
            Set<String> next = new HashSet<>();
            wordSet.removeAll(level);
            for (String word : level) {
                char[] chars = word.toCharArray();
                for (int j = 0; j < chars.length; j++) {
                    char original = chars[j];
                    for (char c = 'a'; c <= 'z'; c++) {
                        chars[j] = c;
                        String candidate = new String(chars);
                        if (wordSet.contains(candidate)) {
                            if (candidate.equals(endWord)) found = true;
                            next.add(candidate);
                            parents.computeIfAbsent(candidate, x -> new ArrayList<>()).add(word);
                        }
                    }
                    chars[j] = original;
                }
            }
            level = next;
        }
        if (found) {
            LinkedList<String> path = new LinkedList<>();
            path.add(endWord);
            backtrack(endWord, beginWord, parents, path, result);
        }
        return result;
    }

    private void backtrack(String word, String beginWord, Map<String, List<String>> parents,
                           LinkedList<String> path, List<List<String>> result) {
        if (word.equals(beginWord)) {
            result.add(new ArrayList<>(path));
            return;
        }
        if (!parents.containsKey(word)) return;
        for (String parent : parents.get(word)) {
            path.addFirst(parent);
            backtrack(parent, beginWord, parents, path, result);
            path.removeFirst();
        }
    }
}

```

**Time**  $O(N \cdot L \cdot 26)$  BFS + paths | **Space**  $O(N \cdot L)$

## #133 Clone Graph

**Description:** Deep-copy an undirected graph where each node has a value and a list of neighbors.

**Intuition:** Use a map from original node to its clone to avoid re-cloning and to wire neighbors correctly; BFS the graph, creating clones on first sight.

**Algorithm:** BFS with a `Map<Node, Node>` of original→clone; for each node, attach cloned neighbors.

```

/*
// Definition for a Node.
class Node {
    public int val;
    public List<Node> neighbors;
    public Node() { val = 0; neighbors = new ArrayList<Node>(); }
    public Node(int _val) { val = _val; neighbors = new ArrayList<Node>(); }
    public Node(int _val, ArrayList<Node> _neighbors) { val = _val; neighbors = _neighbors; }
}
*/
class Solution {
    public Node cloneGraph(Node node) {
        if (node == null) return null;
        Map<Node, Node> clones = new HashMap<>();
        Queue<Node> queue = new ArrayDeque<>();
        queue.offer(node);
        clones.put(node, new Node(node.val));
        while (!queue.isEmpty()) {
            Node current = queue.poll();
            for (Node neighbor : current.neighbors) {
                if (!clones.containsKey(neighbor)) {
                    clones.put(neighbor, new Node(neighbor.val));
                    queue.offer(neighbor);
                }
                clones.get(current).neighbors.add(clones.get(neighbor));
            }
        }
        return clones.get(node);
    }
}

```

**Time**  $O(V+E)$  | **Space**  $O(V)$

## #277 Find the Celebrity

**Description:** Among  $n$  people, find the celebrity: known by everyone, knows nobody. `knows(a, b)` is the API.

**Intuition:** A single linear scan narrows to one candidate (if `candidate` knows `i`, the candidate can't be the celebrity, so `i` becomes the new candidate); a second pass verifies.

**Algorithm:** Two passes — find candidate, then validate both directions.

```

/* The knows API is defined in the parent class Relation.
boolean knows(int a, int b); */
public class Solution extends Relation {
    public int findCelebrity(int n) {
        int candidate = 0;
        for (int i = 1; i < n; i++) {
            if (knows(candidate, i)) {
                candidate = i;
            }
        }
        for (int i = 0; i < n; i++) {
            if (i == candidate) continue;
            if (knows(candidate, i) || !knows(i, candidate)) return -1;
        }
        return candidate;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$

## #296 Best Meeting Point

**Description:** Given a grid where 1 marks a home, find the minimum total Manhattan distance to a single meeting point.

**Intuition:** Manhattan distance separates into independent x and y components; the optimal coordinate on each axis is the median of the occupied coordinates.

**Algorithm:** Collect row and column indices (rows in sorted order, columns sorted), take the median on each axis, sum absolute distances.

```
class Solution {
    public int minTotalDistance(int[][] grid) {
        int rows = grid.length, cols = grid[0].length;
        List<Integer> rowList = new ArrayList<>();
        List<Integer> colList = new ArrayList<>();
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                if (grid[i][j] == 1) {
                    rowList.add(i);
                    colList.add(j);
                }
            }
        }
        Collections.sort(colList);
        int total = 0;
        int medianRow = rowList.get(rowList.size() / 2);
        int medianCol = colList.get(colList.size() / 2);
        for (int r : rowList) {
            total += Math.abs(r - medianRow);
        }
        for (int c : colList) {
            total += Math.abs(c - medianCol);
        }
        return total;
    }
}
```

**Time**  $O(m \cdot n)$  | **Space**  $O(m \cdot n)$

## #305 Number of Islands II

**Description:** Land cells are added one at a time to a water grid; after each addition return the current number of islands.

**Variation:** Online Union-Find — when a cell becomes land, union it with any already-land 4-neighbors; track component count.

```

class Solution {
    int[] parent, rank;
    public List<Integer> numIslands2(int m, int n, int[][] positions) {
        parent = new int[m * n];
        rank = new int[m * n];
        Arrays.fill(parent, -1); // ← VARIATION: -1 marks water (not yet l
        int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};
        List<Integer> result = new ArrayList<>();
        int count = 0;
        for (int[] p : positions) {
            int i = p[0], j = p[1], id = i * n + j;
            if (parent[id] != -1) { // already land
                result.add(count);
                continue;
            }
            parent[id] = id;
            count++;
            for (int dir = 0; dir < 4; dir++) {
                int ni = i + dr[dir], nj = j + dc[dir];
                if (ni < 0 || ni >= m || nj < 0 || nj >= n) continue;
                int nid = ni * n + nj;
                if (parent[nid] != -1 && union(id, nid)) count--;
            }
            result.add(count);
        }
        return result;
    }
    int find(int x) { return parent[x] == x ? x : (parent[x] = find(parent[x])); }
    boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;
        if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
        parent[py] = px;
        if (rank[px] == rank[py]) rank[px]++;
        return true;
    }
}

```

**Time**  $O(k \cdot \alpha(m \cdot n))$  | **Space**  $O(m \cdot n)$

## #317 Shortest Distance from All Buildings

**Description:** Find the empty land cell 0 minimizing total walking distance to every building 1 (2 = obstacle).

**Intuition:** BFS from each building, accumulating distances into every reachable empty cell and counting how many buildings reach it; the answer is the minimum total among cells reachable by all buildings.

**Algorithm:** Multi-pass BFS — one BFS per building; track `total[][]` distance sum and `reach[][]` count.

```

class Solution {
    public int shortestDistance(int[][] grid) {
        int rows = grid.length, cols = grid[0].length;
        int[][] total = new int[rows][cols];
        int[][] reach = new int[rows][cols];
        int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};
        int buildings = 0;
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                if (grid[i][j] != 1) continue;
                buildings++;
                Queue<int[]> queue = new ArrayDeque<>();
                queue.offer(new int[]{i, j});
                boolean[][] visited = new boolean[rows][cols];
                visited[i][j] = true;
                int dist = 0;
                while (!queue.isEmpty()) {
                    int size = queue.size();
                    dist++;
                    for (int s = 0; s < size; s++) {
                        int[] current = queue.poll();
                        for (int dir = 0; dir < 4; dir++) {
                            int ni = current[0] + dr[dir], nj = current[1] + dc[dir];
                            if (ni < 0 || ni >= rows || nj < 0 || nj >= cols) continue;
                            if (visited[ni][nj] || grid[ni][nj] != 0) continue;
                            visited[ni][nj] = true;
                            total[ni][nj] += dist;
                            reach[ni][nj]++;
                            queue.offer(new int[]{ni, nj});
                        }
                    }
                }
            }
        }
        int result = Integer.MAX_VALUE;
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                if (grid[i][j] == 0 && reach[i][j] == buildings) {
                    result = Math.min(result, total[i][j]);
                }
            }
        }
        return result == Integer.MAX_VALUE ? -1 : result;
    }
}

```

**Time**  $O(B \cdot m \cdot n)$  where  $B$  = buildings | **Space**  $O(m \cdot n)$

## #399 Evaluate Division

**Description:** Given equations  $a/b = \text{value}$ , answer queries  $c/d$ . Return -1.0 if undeterminable.

**Intuition:** Treat variables as nodes and ratios as weighted directed edges; the answer to a query is the product of edge weights along any path from numerator to denominator.

**Algorithm:** Build weighted graph, then DFS each query multiplying weights along the path.

```

class Solution {
    public double[] calcEquation(List<List<String>> equations, double[] values, List<List<String>> queries) {
        Map<String, Map<String, Double>> graph = new HashMap<>();
        for (int i = 0; i < equations.size(); i++) {
            String a = equations.get(i).get(0), b = equations.get(i).get(1);
            graph.computeIfAbsent(a, x -> new HashMap<>()).put(b, values[i]);
            graph.computeIfAbsent(b, x -> new HashMap<>()).put(a, 1.0 / values[i]);
        }
        double[] result = new double[queries.size()];
        for (int i = 0; i < queries.size(); i++) {
            String src = queries.get(i).get(0), dst = queries.get(i).get(1);
            if (!graph.containsKey(src) || !graph.containsKey(dst)) {
                result[i] = -1.0;
            } else {
                result[i] = dfs(graph, src, dst, 1.0, new HashSet<>());
            }
        }
        return result;
    }
    private double dfs(Map<String, Map<String, Double>> graph, String node, String target,
                       double product, Set<String> visited) {
        if (node.equals(target)) return product;
        visited.add(node);
        for (Map.Entry<String, Double> nb : graph.get(node).entrySet()) {
            if (visited.contains(nb.getKey())) continue;
            double sub = dfs(graph, nb.getKey(), target, product * nb.getValue(), visited);
            if (sub != -1.0) return sub;
        }
        return -1.0;
    }
}

```

**Time**  $O(Q \cdot (V+E))$  | **Space**  $O(V+E)$

## #407 Trapping Rain Water II

**Description:** Given a 2D height map, compute how much water it can trap after raining.

**Variation:** Dijkstra-style BFS from the border inward using a min-heap of boundary heights; water level at a cell is the highest of the minimum boundary it must pass — pop the lowest wall first.

```

class Solution {
    public int trapRainWater(int[][] heightMap) {
        int rows = heightMap.length, cols = heightMap[0].length;
        if (rows < 3 || cols < 3) return 0;
        boolean[][] visited = new boolean[rows][cols];
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[2] - b[2]); // ← VARIATION: heap on c
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                if (i == 0 || i == rows - 1 || j == 0 || j == cols - 1) {
                    pq.offer(new int[]{i, j, heightMap[i][j]});
                    visited[i][j] = true;
                }
            }
        }
        int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};
        int water = 0;
        while (!pq.isEmpty()) {
            int[] current = pq.poll();
            int level = current[2];
            for (int dir = 0; dir < 4; dir++) {
                int ni = current[0] + dr[dir], nj = current[1] + dc[dir];
                if (ni < 0 || ni >= rows || nj < 0 || nj >= cols || visited[ni][nj]) continue;
                visited[ni][nj] = true;
                if (heightMap[ni][nj] < level) water += level - heightMap[ni][nj];
                pq.offer(new int[]{ni, nj, Math.max(level, heightMap[ni][nj])});
            }
        }
        return water;
    }
}

```

**Time**  $O(m \cdot n \cdot \log(m \cdot n))$  | **Space**  $O(m \cdot n)$

## #463 Island Perimeter

**Description:** Given a grid with exactly one island, return its perimeter.

**Intuition:** Each land cell contributes 4 sides, minus 2 for every shared edge with an adjacent land cell (counted once per pair).

**Algorithm:** Count land cells  $\times 4$ , subtract 2 for each adjacent land pair.

```

class Solution {
    public int islandPerimeter(int[][] grid) {
        int rows = grid.length, cols = grid[0].length;
        int perimeter = 0;
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                if (grid[i][j] == 1) {
                    perimeter += 4;
                    if (i > 0 && grid[i-1][j] == 1) perimeter -= 2;
                    if (j > 0 && grid[i][j-1] == 1) perimeter -= 2;
                }
            }
        }
        return perimeter;
    }
}

```

**Time**  $O(m \cdot n)$  | **Space**  $O(1)$

## #529 Minesweeper

**Description:** Reveal a cell on a Minesweeper board. 'M' mine, 'E' unrevealed empty, 'B' revealed blank, digit = adjacent mine count.

**Variation:** BFS/DFS flood fill with 8 directions; if the clicked cell is a mine, mark 'X'; otherwise count adjacent mines, and only recurse when count is 0.

```
class Solution {
    public char[][] updateBoard(char[][] board, int[] click) {
        int rows = board.length, cols = board[0].length;
        int cr = click[0], cc = click[1];
        if (board[cr][cc] == 'M') {
            board[cr][cc] = 'X';
            return board;
        }
        int[] dr = {1,-1,0,0,1,1,-1,-1}, dc = {0,0,1,-1,1,-1,1,-1}; // ← VARIATION: 8 directions
        Queue<int[]> queue = new ArrayDeque<>();
        queue.offer(new int[]{cr, cc});
        boolean[][] visited = new boolean[rows][cols];
        visited[cr][cc] = true;
        while (!queue.isEmpty()) {
            int[] current = queue.poll();
            int r = current[0], c = current[1];
            int mines = 0;
            for (int dir = 0; dir < 8; dir++) {
                int nr = r + dr[dir], nc = c + dc[dir];
                if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && board[nr][nc] == 'M') mines++;
            }
            if (mines > 0) {
                board[r][c] = (char) ('0' + mines);
            } else {
                board[r][c] = 'B';
                for (int dir = 0; dir < 8; dir++) {
                    int nr = r + dr[dir], nc = c + dc[dir];
                    if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;
                    if (!visited[nr][nc] && board[nr][nc] == 'E') {
                        visited[nr][nc] = true;
                        queue.offer(new int[]{nr, nc});
                    }
                }
            }
        }
        return board;
    }
}
```

**Time**  $O(m \cdot n)$  | **Space**  $O(m \cdot n)$

## #694 Number of Distinct Islands

**Description:** Count islands that are distinct by shape (translations are the same, rotations are not).

**Variation:** DFS recording the path signature (the direction taken at each step plus a backtrack marker); two islands with identical signatures have the same shape.



```

class Solution {
    public int numDistinctIslands(int[][] grid) {
        int rows = grid.length, cols = grid[0].length;
        Set<String> shapes = new HashSet<>();
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                if (grid[i][j] == 1) {
                    StringBuilder builder = new StringBuilder();
                    dfs(grid, i, j, 'o', builder);           // ← VARIATION: record traversal path
                    shapes.add(builder.toString());
                }
            }
        }
        return shapes.size();
    }
    private void dfs(int[][] grid, int i, int j, char dir, StringBuilder builder) {
        if (i < 0 || i >= grid.length || j < 0 || j >= grid[0].length || grid[i][j] != 1) return;
        grid[i][j] = 0;
        builder.append(dir);
        dfs(grid, i + 1, j, 'd', builder);
        dfs(grid, i - 1, j, 'u', builder);
        dfs(grid, i, j + 1, 'r', builder);
        dfs(grid, i, j - 1, 'l', builder);
        builder.append('b');                               // ← VARIATION: backtrack marker disamb
    }
}

```

**Time**  $O(m \cdot n)$  | **Space**  $O(m \cdot n)$

## #733 Flood Fill

**Description:** Starting at  $(sr, sc)$ , change all connected same-color pixels to  $color$ .

**Algorithm:** BFS/DFS flood fill from the start cell over 4 directions; guard against re-filling when the new color equals the original.

```

class Solution {
    public int[][] floodFill(int[][] image, int sr, int sc, int color) {
        int original = image[sr][sc];
        if (original == color) return image;
        int rows = image.length, cols = image[0].length;
        int[] dr = {1, -1, 0, 0}, dc = {0, 0, 1, -1};
        Queue<int[]> queue = new ArrayDeque<>();
        queue.offer(new int[]{sr, sc});
        image[sr][sc] = color;
        while (!queue.isEmpty()) {
            int[] current = queue.poll();
            for (int dir = 0; dir < 4; dir++) {
                int ni = current[0] + dr[dir], nj = current[1] + dc[dir];
                if (ni < 0 || ni >= rows || nj < 0 || nj >= cols) continue;
                if (image[ni][nj] == original) {
                    image[ni][nj] = color;
                    queue.offer(new int[]{ni, nj});
                }
            }
        }
        return image;
    }
}

```

**Time**  $O(m \cdot n)$  | **Space**  $O(m \cdot n)$

## #752 Open the Lock

**Description:** A 4-wheel lock starts at "0000"; each move turns one wheel  $\pm 1$ . Find minimum moves to reach `target`, avoiding `deadends`.

**Variation:** BFS over the 4-digit state space; each state has 8 neighbors (each wheel up or down). Treat deadends as visited.

```
class Solution {
    public int openLock(String[] deadends, String target) {
        Set<String> dead = new HashSet<>(Arrays.asList(deadends));
        if (dead.contains("0000")) return -1;
        if (target.equals("0000")) return 0;
        Set<String> visited = new HashSet<>();
        visited.add("0000");
        Queue<String> queue = new ArrayDeque<>();
        queue.offer("0000");
        int turns = 0;
        while (!queue.isEmpty()) {
            int size = queue.size();
            turns++;
            for (int s = 0; s < size; s++) {
                String current = queue.poll();
                for (int j = 0; j < 4; j++) { // ← VARIATION: each wheel  $\pm 1$  = 8 neighb
                    char c = current.charAt(j);
                    for (int delta = -1; delta <= 1; delta += 2) {
                        char nc = (char) ('0' + (c - '0' + delta + 10) % 10);
                        String next = current.substring(0, j) + nc + current.substring(j + 1);
                        if (next.equals(target)) return turns;
                        if (!dead.contains(next) && visited.add(next)) queue.offer(next);
                    }
                }
            }
        }
        return -1;
    }
}
```

**Time**  $O(10^4 \cdot 8)$  | **Space**  $O(10^4)$

## #778 Swim in Rising Water

**Description:** At time `t` you can stand on any cell with elevation  $\leq t$ . Find the least time to swim from  $(0, 0)$  to  $(n-1, n-1)$ .

**Variation:** Modified Dijkstra — minimize the maximum elevation along the path (not the sum). Min-heap ordered by current max elevation.

```

class Solution {
    public int swimInWater(int[][] grid) {
        int n = grid.length;
        boolean[][] visited = new boolean[n][n];
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]); // [maxElev, r, c]
        pq.offer(new int[]{grid[0][0], 0, 0});
        visited[0][0] = true;
        int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};
        while (!pq.isEmpty()) {
            int[] current = pq.poll();
            int time = current[0], r = current[1], c = current[2];
            if (r == n - 1 && c == n - 1) return time;
            for (int dir = 0; dir < 4; dir++) {
                int nr = r + dr[dir], nc = c + dc[dir];
                if (nr < 0 || nr >= n || nc < 0 || nc >= n || visited[nr][nc]) continue;
                visited[nr][nc] = true;
                pq.offer(new int[]{Math.max(time, grid[nr][nc]), nr, nc}); // ← VARIATION: max not sum
            }
        }
        return -1;
    }
}

```

**Time**  $O(n^2 \cdot \log(n))$  | **Space**  $O(n^2)$

## #797 All Paths From Source to Target

**Description:** In a DAG given as adjacency list `graph`, return all paths from node `0` to node `n-1`.

**Intuition:** Since it is a DAG with no cycles, plain DFS backtracking enumerates every path; no visited set is needed.

**Algorithm:** DFS from `0`, appending nodes to a path, recording the path when reaching `n-1`.

```

class Solution {
    public List<List<Integer>> allPathsSourceTarget(int[][] graph) {
        List<List<Integer>> result = new ArrayList<>();
        List<Integer> path = new ArrayList<>();
        path.add(0);
        dfs(graph, 0, path, result);
        return result;
    }
    private void dfs(int[][] graph, int node, List<Integer> path, List<List<Integer>> result) {
        if (node == graph.length - 1) {
            result.add(new ArrayList<>(path));
            return;
        }
        for (int neighbor : graph[node]) {
            path.add(neighbor);
            dfs(graph, neighbor, path, result);
            path.remove(path.size() - 1);
        }
    }
}

```

**Time**  $O(2^V \cdot V)$  | **Space**  $O(V)$

## #802 Find Eventual Safe States

**Description:** A node is safe if every path from it eventually reaches a terminal node (no outgoing edges). Return all safe nodes sorted.

**Variation:** Topological sort on the reversed graph (Kahn's) — start from terminal nodes (out-degree 0); a node becomes safe once all its outgoing edges lead to safe nodes.

```
class Solution {
    public List<Integer> eventualSafeNodes(int[][] graph) {
        int n = graph.length;
        List<List<Integer>> reverse = new ArrayList<>();
        for (int i = 0; i < n; i++) reverse.add(new ArrayList<>());
        int[] outDegree = new int[n]; // ← VARIATION: track outgoing count
        for (int i = 0; i < n; i++) {
            outDegree[i] = graph[i].length;
            for (int neighbor : graph[i]) reverse.get(neighbor).add(i);
        }
        Queue<Integer> queue = new ArrayDeque<>();
        for (int i = 0; i < n; i++) if (outDegree[i] == 0) queue.offer(i);
        boolean[] safe = new boolean[n];
        while (!queue.isEmpty()) {
            int node = queue.poll();
            safe[node] = true;
            for (int prev : reverse.get(node)) {
                if (--outDegree[prev] == 0) queue.offer(prev);
            }
        }
        List<Integer> result = new ArrayList<>();
        for (int i = 0; i < n; i++) if (safe[i]) result.add(i);
        return result;
    }
}
```

**Time**  $O(V+E)$  | **Space**  $O(V+E)$

## #815 Bus Routes

**Description:** `routes[i]` is the stops served by bus `i`. Find the minimum number of buses to travel from `source` to `target`.

**Variation:** BFS where the "nodes" are buses (routes); build stop→routes index, then BFS route-to-route through shared stops, counting buses taken.

```

class Solution {
    public int numBusesToDestination(int[][] routes, int source, int target) {
        if (source == target) return 0;
        Map<Integer, List<Integer>> stopToRoutes = new HashMap<>();
        for (int i = 0; i < routes.length; i++) {
            for (int stop : routes[i]) {
                stopToRoutes.computeIfAbsent(stop, x -> new ArrayList<>()).add(i);
            }
        }
        Queue<Integer> queue = new ArrayDeque<>(); // ← VARIATION: queue holds stops
        Set<Integer> visitedStops = new HashSet<>();
        boolean[] visitedRoutes = new boolean[routes.length];
        queue.offer(source);
        visitedStops.add(source);
        int buses = 0;
        while (!queue.isEmpty()) {
            int size = queue.size();
            buses++;
            for (int s = 0; s < size; s++) {
                int stop = queue.poll();
                for (int route : stopToRoutes.getOrDefault(stop, Collections.emptyList())) {
                    if (visitedRoutes[route]) continue;
                    visitedRoutes[route] = true;
                    for (int nextStop : routes[route]) {
                        if (nextStop == target) return buses;
                        if (visitedStops.add(nextStop)) queue.offer(nextStop);
                    }
                }
            }
        }
        return -1;
    }
}

```

**Time**  $O(\text{sum of route lengths})$  | **Space**  $O(\text{sum of route lengths})$

## #847 Shortest Path Visiting All Nodes

**Description:** Find the length of the shortest path in an undirected graph that visits every node (may revisit nodes/edges).

**Variation:** BFS over states  $(\text{node}, \text{visitedMask})$  where  $\text{mask}$  is a bitmask of visited nodes; the goal is any state with all bits set. Start BFS from every node simultaneously.

```

class Solution {
    public int shortestPathLength(int[][] graph) {
        int n = graph.length;
        int allVisited = (1 << n) - 1;
        Queue<int[]> queue = new ArrayDeque<>(); // [node, mask]
        boolean[][] visited = new boolean[n][1 << n];
        for (int i = 0; i < n; i++) { // ← VARIATION: seed all nodes
            queue.offer(new int[]{i, 1 << i});
            visited[i][1 << i] = true;
        }
        int steps = 0;
        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int s = 0; s < size; s++) {
                int[] current = queue.poll();
                int node = current[0], mask = current[1];
                if (mask == allVisited) return steps;
                for (int neighbor : graph[node]) {
                    int nextMask = mask | (1 << neighbor);
                    if (!visited[neighbor][nextMask]) {
                        visited[neighbor][nextMask] = true;
                        queue.offer(new int[]{neighbor, nextMask});
                    }
                }
            }
            steps++;
        }
        return 0;
    }
}

```

**Time**  $O(2^n \cdot n^2)$  | **Space**  $O(2^n \cdot n)$

## #851 Loud and Rich

**Description:** `richer[i] = [a, b]` means `a` is richer than `b`; `quiet[i]` is the quietness of person `i`. For each person find the quietest person among everyone at least as rich (themselves included).

**Variation:** Topological sort (Kahn's) on the richer→poorer graph; propagate the quietest-known answer from richer people down to poorer ones in topo order.

```

class Solution {
    public int[] loudAndRich(int[][] richer, int[] quiet) {
        int n = quiet.length;
        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
        int[] inDegree = new int[n];
        for (int[] r : richer) {
            graph.get(r[0]).add(r[1]);
            inDegree[r[1]]++;
        }
        int[] result = new int[n];
        for (int i = 0; i < n; i++) result[i] = i;
        Queue<Integer> queue = new ArrayDeque<>();
        for (int i = 0; i < n; i++) if (inDegree[i] == 0) queue.offer(i);
        while (!queue.isEmpty()) {
            int node = queue.poll();
            for (int neighbor : graph.get(node)) {
                if (quiet[result[node]] < quiet[result[neighbor]]) {
                    result[neighbor] = result[node];
                    // ← VARIATION: propagate quietest down
                }
                if (--inDegree[neighbor] == 0) queue.offer(neighbor);
            }
        }
        return result;
    }
}

```

**Time**  $O(V+E)$  | **Space**  $O(V+E)$

## #886 Possible Bipartition

**Description:** Given `dislikes` pairs, split `n` people into two groups so disliking people are never in the same group.

**Algorithm:** Bipartite 2-coloring via BFS over the dislike graph; a same-color conflict means impossible.

```

class Solution {
    public boolean possibleBipartition(int n, int[][] dislikes) {
        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i <= n; i++) graph.add(new ArrayList<>());
        for (int[] d : dislikes) {
            graph.get(d[0]).add(d[1]);
            graph.get(d[1]).add(d[0]);
        }
        int[] color = new int[n + 1];
        Arrays.fill(color, -1);
        Queue<Integer> queue = new ArrayDeque<>();
        for (int start = 1; start <= n; start++) {
            if (color[start] != -1) continue;
            color[start] = 0;
            queue.offer(start);
            while (!queue.isEmpty()) {
                int node = queue.poll();
                for (int neighbor : graph.get(node)) {
                    if (color[neighbor] == -1) {
                        color[neighbor] = 1 - color[node];
                        queue.offer(neighbor);
                    } else if (color[neighbor] == color[node]) {
                        return false;
                    }
                }
            }
        }
        return true;
    }
}

```

**Time**  $O(V+E)$  | **Space**  $O(V+E)$

## #913 Cat and Mouse

**Description:** A mouse (start node 1) and cat (start node 2) move on a graph; mouse wins by reaching node 0, cat wins by catching the mouse, else draw. Return 0/1/2 with optimal play.

**Variation:** Game-theory BFS over states (mouse, cat, turn); start from known terminal states and propagate results backward (retrograde analysis) by counting undecided moves.



```

class Solution {
    public int catMouseGame(int[][] graph) {
        int n = graph.length;
        final int DRAW = 0, MOUSE = 1, CAT = 2;
        int[][][] result = new int[n][n][2]; // [mouse][cat][turn] : 0=mouse turn,1=c
        int[][][] degree = new int[n][n][2];
        Queue<int[]> queue = new ArrayDeque<>();
        for (int m = 0; m < n; m++) {
            for (int c = 0; c < n; c++) {
                degree[m][c][0] = graph[m].length;
                degree[m][c][1] = graph[c].length;
                for (int x : graph[c]) {
                    if (x == 0) { degree[m][c][1]--; break; } // cat cannot move into hole 0
                }
            }
        }
        for (int c = 0; c < n; c++) {
            for (int t = 0; t < 2; t++) {
                result[0][c][t] = MOUSE; // mouse at hole → mouse wins
                queue.offer(new int[]{0, c, t, MOUSE});
                if (c > 0) {
                    result[c][c][t] = CAT; // cat catches mouse
                    queue.offer(new int[]{c, c, t, CAT});
                }
            }
        }
        while (!queue.isEmpty()) {
            int[] state = queue.poll();
            int mouse = state[0], cat = state[1], turn = state[2], winner = state[3];
            for (int[] prev : parents(graph, mouse, cat, turn)) {
                int pm = prev[0], pc = prev[1], pt = prev[2];
                if (result[pm][pc][pt] != DRAW) continue;
                if ((pt == 0 && winner == MOUSE) || (pt == 1 && winner == CAT)) {
                    result[pm][pc][pt] = winner; // mover can force its own win
                    queue.offer(new int[]{pm, pc, pt, winner});
                } else if (--degree[pm][pc][pt] == 0) { // ← VARIATION: all moves lead to oppon
                    result[pm][pc][pt] = winner;
                    queue.offer(new int[]{pm, pc, pt, winner});
                }
            }
        }
        return result[1][2][0];
    }
    private List<int[]> parents(int[][] graph, int mouse, int cat, int turn) {
        List<int[]> result = new ArrayList<>();
        if (turn == 0) { // previous move was the cat's (turn 1)
            for (int pc : graph[cat]) {
                if (pc != 0) result.add(new int[]{mouse, pc, 1});
            }
        } else { // previous move was the mouse's (turn
            for (int pm : graph[mouse]) {
                result.add(new int[]{pm, cat, 0});
            }
        }
        return result;
    }
}

```

**Time**  $O(V^3)$  | **Space**  $O(V^2)$

## #934 Shortest Bridge

**Description:** A grid has exactly two islands of 1 s. Find the minimum number of 0 s to flip to connect them.

**Variation:** DFS to mark the first island (seeding a BFS queue with its cells), then multi-source BFS expanding outward until reaching the second island; BFS levels = bridge length.

```
class Solution {
    public int shortestBridge(int[][] grid) {
        int n = grid.length;
        int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};
        Queue<int[]> queue = new ArrayDeque<>();
        boolean found = false;
        for (int i = 0; i < n && !found; i++) {
            for (int j = 0; j < n && !found; j++) {
                if (grid[i][j] == 1) {
                    dfs(grid, i, j, queue);
                    found = true;
                }
            }
        }
        int steps = 0;
        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int s = 0; s < size; s++) {
                int[] current = queue.poll();
                for (int dir = 0; dir < 4; dir++) {
                    int ni = current[0] + dr[dir], nj = current[1] + dc[dir];
                    if (ni < 0 || ni >= n || nj < 0 || nj >= n) continue;
                    if (grid[ni][nj] == 1) return steps;
                    if (grid[ni][nj] == 0) {
                        grid[ni][nj] = 2;
                        queue.offer(new int[]{ni, nj});
                    }
                }
            }
            steps++;
        }
        return -1;
    }
    private void dfs(int[][] grid, int i, int j, Queue<int[]> queue) {
        int n = grid.length;
        if (i < 0 || i >= n || j < 0 || j >= n || grid[i][j] != 1) return;
        grid[i][j] = 2;
        queue.offer(new int[]{i, j});
        int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};
        for (int dir = 0; dir < 4; dir++) dfs(grid, i + dr[dir], j + dc[dir], queue);
    }
}
```

**Time**  $O(n^2)$  | **Space**  $O(n^2)$

## #1034 Coloring A Border

**Description:** Color the border cells of the connected component containing (row, col) with color. A border cell touches the grid edge or a cell of a different original color.

**Variation:** BFS over same-color connected cells; mark a cell as a border when it has fewer than 4 same-component neighbors, recolor borders after traversal.

```

class Solution {
    public int[][] colorBorder(int[][] grid, int row, int col, int color) {
        int rows = grid.length, cols = grid[0].length;
        int original = grid[row][col];
        boolean[][] visited = new boolean[rows][cols];
        List<int[]> borders = new ArrayList<>();
        int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};
        Queue<int[]> queue = new ArrayDeque<>();
        queue.offer(new int[]{row, col});
        visited[row][col] = true;
        while (!queue.isEmpty()) {
            int[] current = queue.poll();
            int r = current[0], c = current[1];
            int sameNeighbors = 0;
            for (int dir = 0; dir < 4; dir++) {
                int nr = r + dr[dir], nc = c + dc[dir];
                if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;
                if (grid[nr][nc] == original) sameNeighbors++;
                if (grid[nr][nc] == original && !visited[nr][nc]) {
                    visited[nr][nc] = true;
                    queue.offer(new int[]{nr, nc});
                }
            }
            if (sameNeighbors < 4) borders.add(new int[]{r, c}); // ← VARIATION: fewer than 4 component
        }
        for (int[] b : borders) grid[b[0]][b[1]] = color;
        return grid;
    }
}

```

**Time**  $O(m \cdot n)$  | **Space**  $O(m \cdot n)$

## #1102 Path With Maximum Minimum Value

**Description:** Find a path from top-left to bottom-right maximizing the minimum cell value along the path (4-directional).

**Variation:** Modified Dijkstra with a max-heap — the path "score" is the minimum cell value seen; greedily expand the highest-scoring frontier first.

```

class Solution {
    public int maximumMinimumPath(int[][] grid) {
        int rows = grid.length, cols = grid[0].length;
        boolean[][] visited = new boolean[rows][cols];
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> b[0] - a[0]); // ← VARIATION: max-heap
        pq.offer(new int[]{grid[0][0], 0, 0});
        visited[0][0] = true;
        int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};
        while (!pq.isEmpty()) {
            int[] current = pq.poll();
            int score = current[0], r = current[1], c = current[2];
            if (r == rows - 1 && c == cols - 1) return score;
            for (int dir = 0; dir < 4; dir++) {
                int nr = r + dr[dir], nc = c + dc[dir];
                if (nr < 0 || nr >= rows || nc < 0 || nc >= cols || visited[nr][nc]) continue;
                visited[nr][nc] = true;
                pq.offer(new int[]{Math.min(score, grid[nr][nc]), nr, nc}); // ← VARIATION: min not sum
            }
        }
        return -1;
    }
}

```

**Time**  $O(m \cdot n \cdot \log(m \cdot n))$  | **Space**  $O(m \cdot n)$

## #1136 Parallel Courses

**Description:** Courses  $1..n$  with prerequisite relations; in each semester take all courses whose prerequisites are done. Return minimum semesters, or -1 if impossible.

**Variation:** Topological sort (Kahn's) processed level-by-level — each BFS level is one semester; if not all courses are processed, a cycle exists.

```
class Solution {
    public int minimumSemesters(int n, int[][] relations) {
        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i <= n; i++) graph.add(new ArrayList<>());
        int[] inDegree = new int[n + 1];
        for (int[] r : relations) {
            graph.get(r[0]).add(r[1]);
            inDegree[r[1]]++;
        }
        Queue<Integer> queue = new ArrayDeque<>();
        for (int i = 1; i <= n; i++) if (inDegree[i] == 0) queue.offer(i);
        int semesters = 0, studied = 0;
        while (!queue.isEmpty()) {
            int size = queue.size();
            semesters++; // ← VARIATION: each level = one semest
            for (int s = 0; s < size; s++) {
                int node = queue.poll();
                studied++;
                for (int neighbor : graph.get(node)) {
                    if (--inDegree[neighbor] == 0) queue.offer(neighbor);
                }
            }
        }
        return studied == n ? semesters : -1;
    }
}
```

**Time**  $O(V+E)$  | **Space**  $O(V+E)$

## #1162 As Far from Land as Possible

**Description:** In a grid of 0 (water) and 1 (land), find the water cell whose distance to the nearest land is maximized; return that distance, or -1.

**Variation:** Multi-source BFS seeded with all land cells; the last BFS level expanded gives the maximum distance.

```

class Solution {
    public int maxDistance(int[][] grid) {
        int n = grid.length;
        Queue<int[]> queue = new ArrayDeque<>();
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (grid[i][j] == 1) queue.offer(new int[]{i, j}); // ← VARIATION: seed ALL land cells
            }
        }
        if (queue.isEmpty() || queue.size() == n * n) return -1;
        int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};
        int distance = -1;
        while (!queue.isEmpty()) {
            int size = queue.size();
            distance++;
            for (int s = 0; s < size; s++) {
                int[] current = queue.poll();
                for (int dir = 0; dir < 4; dir++) {
                    int ni = current[0] + dr[dir], nj = current[1] + dc[dir];
                    if (ni < 0 || ni >= n || nj < 0 || nj >= n || grid[ni][nj] != 0) continue;
                    grid[ni][nj] = 1;
                    queue.offer(new int[]{ni, nj});
                }
            }
        }
        return distance;
    }
}

```

**Time**  $O(n^2)$  | **Space**  $O(n^2)$

## #1197 Minimum Knight Moves

**Description:** On an infinite chessboard, find the minimum knight moves from  $(0,0)$  to  $(x, y)$ .

**Variation:** BFS over knight moves (8 jump offsets); exploit symmetry by working in the first quadrant to bound the visited set.

```

class Solution {
    public int minKnightMoves(int x, int y) {
        x = Math.abs(x);
        y = Math.abs(y); // ← VARIATION: symmetry to first quadr
        int[] dr = {1,1,-1,-1,2,2,-2,-2}, dc = {2,-2,2,-2,1,-1,1,-1}; // knight offsets
        Queue<int[]> queue = new ArrayDeque<>();
        queue.offer(new int[]{0, 0});
        Set<String> visited = new HashSet<>();
        visited.add("0,0");
        int moves = 0;
        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int s = 0; s < size; s++) {
                int[] current = queue.poll();
                if (current[0] == x && current[1] == y) return moves;
                for (int dir = 0; dir < 8; dir++) {
                    int ni = current[0] + dr[dir], nj = current[1] + dc[dir];
                    if (ni < -2 || nj < -2) continue; // stay near first quadrant
                    String key = ni + "," + nj;
                    if (visited.add(key)) queue.offer(new int[]{ni, nj});
                }
            }
            moves++;
        }
        return -1;
    }
}

```

**Time**  $O(\max(x,y)^2)$  | **Space**  $O(\max(x,y)^2)$

## #1202 Smallest String With Swaps

**Description:** Given index `pairs` that may be swapped any number of times, return the lexicographically smallest string achievable.

**Variation:** Union-Find groups swappable indices into connected components; within each component, sort the characters and place them back in ascending index order.

```

class Solution {
    int[] parent, rank;
    public String smallestStringWithSwaps(String s, List<List<Integer>> pairs) {
        int n = s.length();
        parent = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
        for (List<Integer> p : pairs) union(p.get(0), p.get(1)); // ← VARIATION: union swappable indices
        Map<Integer, List<Integer>> groups = new HashMap<>();
        for (int i = 0; i < n; i++) {
            groups.computeIfAbsent(find(i), x -> new ArrayList<>()).add(i);
        }
        char[] result = s.toCharArray();
        for (List<Integer> indices : groups.values()) {
            List<Character> chars = new ArrayList<>();
            for (int i : indices) chars.add(s.charAt(i));
            Collections.sort(chars);
            for (int k = 0; k < indices.size(); k++) result[indices.get(k)] = chars.get(k);
        }
        return new String(result);
    }
    int find(int x) { return parent[x] == x ? x : (parent[x] = find(parent[x])); }
    boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;
        if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
        parent[py] = px;
        if (rank[px] == rank[py]) rank[px]++;
        return true;
    }
}

```

**Time**  $O(n \cdot \log(n) + E \cdot \alpha(n))$  | **Space**  $O(n)$

## #1245 Tree Diameter

**Description:** Given the edges of an undirected tree, return its diameter (the number of edges on the longest path between any two nodes).

**Variation:** Two BFS passes — BFS from any node finds the farthest node `u`; a second BFS from `u` gives the diameter as the maximum distance.

```

class Solution {
    public int treeDiameter(int[][] edges) {
        int n = edges.length + 1;
        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
        for (int[] e : edges) {
            graph.get(e[0]).add(e[1]);
            graph.get(e[1]).add(e[0]);
        }
        int[] first = bfs(graph, 0, n); // ← VARIATION: farthest node from arbitrary node
        int[] second = bfs(graph, first[0], n); // diameter from that farthest node
        return second[1];
    }
    private int[] bfs(List<List<Integer>> graph, int start, int n) {
        boolean[] visited = new boolean[n];
        Queue<Integer> queue = new ArrayDeque<>();
        queue.offer(start);
        visited[start] = true;
        int farthest = start, distance = -1;
        while (!queue.isEmpty()) {
            int size = queue.size();
            distance++;
            for (int s = 0; s < size; s++) {
                int node = queue.poll();
                farthest = node;
                for (int neighbor : graph.get(node)) {
                    if (!visited[neighbor]) {
                        visited[neighbor] = true;
                        queue.offer(neighbor);
                    }
                }
            }
        }
        return new int[]{farthest, distance};
    }
}

```

**Time**  $O(V+E)$  | **Space**  $O(V+E)$

## #1293 Shortest Path in a Grid with Obstacles Elimination

**Description:** Find the shortest path from  $(0,0)$  to  $(m-1,n-1)$  in a grid where you may eliminate at most  $k$  obstacles (1 s).

**Variation:** BFS over states  $(row, col, remainingK)$ ; the visited set tracks the best remaining eliminations per cell so a cell can be revisited with more budget.



```

class Solution {
    public int shortestPath(int[][] grid, int k) {
        int rows = grid.length, cols = grid[0].length;
        if (k >= rows + cols - 2) return rows + cols - 2; // enough to go straight
        int[][] bestK = new int[rows][cols];
        for (int[] row : bestK) Arrays.fill(row, -1);
        int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};
        Queue<int[]> queue = new ArrayDeque<>(); // [r, c, remainingK]
        queue.offer(new int[]{0, 0, k});
        bestK[0][0] = k;
        int steps = 0;
        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int s = 0; s < size; s++) {
                int[] current = queue.poll();
                int r = current[0], c = current[1], rem = current[2];
                if (r == rows - 1 && c == cols - 1) return steps;
                for (int dir = 0; dir < 4; dir++) {
                    int nr = r + dr[dir], nc = c + dc[dir];
                    if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;
                    int nextRem = rem - grid[nr][nc];
                    if (nextRem > bestK[nr][nc]) { // ← VARIATION: revisit if more budget
                        bestK[nr][nc] = nextRem;
                        queue.offer(new int[]{nr, nc, nextRem});
                    }
                }
            }
            steps++;
        }
        return -1;
    }
}

```

**Time**  $O(m \cdot n \cdot k)$  | **Space**  $O(m \cdot n)$

## #1559 Detect Cycles in 2D Grid

**Description:** Return true if the grid contains a cycle of length  $\geq 4$  made of the same character.

**Variation:** Union-Find over grid cells — for each cell union it with its up and left same-character neighbors; if a union finds them already connected, a cycle exists.

```

class Solution {
    int[] parent, rank;
    public boolean containsCycle(char[][] grid) {
        int rows = grid.length, cols = grid[0].length;
        parent = new int[rows * cols];
        rank = new int[rows * cols];
        for (int i = 0; i < rows * cols; i++) parent[i] = i;
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                int id = i * cols + j;
                if (i > 0 && grid[i-1][j] == grid[i][j]) {
                    if (!union(id, (i-1) * cols + j)) return true; // ← VARIATION: already connected →
                }
                if (j > 0 && grid[i][j-1] == grid[i][j]) {
                    if (!union(id, i * cols + (j-1))) return true;
                }
            }
        }
        return false;
    }
    int find(int x) { return parent[x] == x ? x : (parent[x] = find(parent[x])); }
    boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;
        if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
        parent[py] = px;
        if (rank[px] == rank[py]) rank[px]++;
        return true;
    }
}

```

**Time**  $O(m \cdot n \cdot \alpha(m \cdot n))$  | **Space**  $O(m \cdot n)$

## #1743 Restore the Array From Adjacent Pairs

**Description:** Given all adjacent pairs of an array (in any order), reconstruct the original array.

**Variation:** Build an adjacency graph; the two endpoints have a single neighbor. Start from an endpoint and walk the path, avoiding the previous node.

```

class Solution {
    public int[] restoreArray(int[][] adjacentPairs) {
        Map<Integer, List<Integer>> graph = new HashMap<>();
        for (int[] pair : adjacentPairs) {
            graph.computeIfAbsent(pair[0], x -> new ArrayList<>()).add(pair[1]);
            graph.computeIfAbsent(pair[1], x -> new ArrayList<>()).add(pair[0]);
        }
        int n = adjacentPairs.length + 1;
        int[] result = new int[n];
        int start = 0;
        for (Map.Entry<Integer, List<Integer>> entry : graph.entrySet()) {
            if (entry.getValue().size() == 1) { // ← VARIATION: endpoint has one neighb
                start = entry.getKey();
                break;
            }
        }
        result[0] = start;
        result[1] = graph.get(start).get(0);
        for (int i = 2; i < n; i++) {
            List<Integer> neighbors = graph.get(result[i-1]);
            result[i] = neighbors.get(0) == result[i-2] ? neighbors.get(1) : neighbors.get(0);
        }
        return result;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(n)$